



КВЕСТ-КУРС: HTML MIDDLE — ПУТЬ АРХИТЕКТОРА

Квест-курс HTML Middle — Путь Архитектора

Модуль 1 - Семантика и структура

Урок 1 - Семантические теги main, article, aside

Шаг 01:

Миссия 1

Лиза делает страницу новостей. Каждый новостной блок — отдельная единица с заголовком и текстом. Должна ли она оборачивать каждый блок в `<article>`?

Подсказка: Если блок может быть прочитан отдельно и логически завершён — это подходящее место для `<article>`.

Ответ:

- **Да, каждый новостной блок** — отдельный `<article>`
- Нет, лучше всё держать в `<main>` без `<article>`

Шаг 02:

Миссия 3

 Сопоставьте советы и примеры правильного использования:

Ответ:

- Только один `<main>`
- **Article** — самостоятельный блок
- `Aside` для доп. контента

- Нельзя вкладывать main в article

1. на странице ровно один тег <main>
2. каждая статья обернута в article
3. реклама или ссылки вынесены в aside
4. main находится снаружи article

Подсказка: Сравните примеры с хорошей структурой.

- **Только один** — на странице ровно один тег
- **Article** — самостоятельный блок — каждая статья обернута в article
- **Aside для доп. контента** — реклама или ссылки вынесены в aside
- **Нельзя вкладывать main в article** — main находится снаружи article

Шаг 03:

Миссия 4

Расположите этапы построения страницы с правильной семантикой:

1. Определить, что является основным контентом
2. Выбрать место вставки боковых блоков (aside)
3. Обернуть основной контент в <main>
4. Обернуть независимые разделы в <article>
5. Проверить, что вложенность корректна

Подсказка: Начинайте с анализа, затем распределяйте теги, и затем проверяйте вложения.

Ответ:

- Определить, что является основным контентом
- Выбрать место вставки боковых блоков (aside)
- Обернуть основной контент в
- Обернуть независимые разделы в
- Проверить, что вложенность корректна

Шаг 04:

Миссия 5

Валентина уже имеет: header, nav, и содержимое. Нужно вставить main, article, aside и footer в правильном порядке внутри body:

1. main
2. article
3. aside
4. footer

Подсказка: main — контейнер главного, aside может идти рядом с article, footer — внизу.

Ответ:

- main
- article
- aside
- footer

Шаг 05:

Миссия 7

Расположите порядок появления блоков на странице с тремя статьями и боковой панелью:

1. main
2. article (статья 1)
3. article (статья 2)
4. article (статья 3)
5. aside
6. footer

Подсказка: aside можно после статей или внутри main, главное — логика расположения.

Ответ:

- main
- article (статья 1)
- article (статья 2)
- article (статья 3)
- aside
- footer

Шаг 06:

Миссия 7

Никита оформляет подвал сайта и хочет вставить туда контент «О сайте». Он думает: может это основной контент? Нужно ли использовать <main>?

Подсказка: Подумайте: будет ли содержимое подвала центральным содержимым страницы?

Ответ:

- Да, использовать <main>
- **Нет, использовать другой блок (например, footer)**

Шаг 07:

Миссия 8

 Соотнесите правила использования семантических тегов и примеры.

Правила:

Ответ:

- Тег <main> должен быть один
- Тег <article> может быть вложенным
- Тег <aside> не должен содержать основной контент
- Тег <main> не может находиться внутри <article>

Примеры:

1. на странице два разных блока <main>
2. боковая колонка с рекламой внутри статьи
3. статья внутри другой статьи
4. главное содержимое вложено внутрь тега article

Подсказка: Проверьте корректность вложения и количество тегов.

- **Тег <main> должен быть один** — на странице два разных блока <main>

- **Тег <article> может быть вложенным** — статья внутри другой статьи
- **Тег <aside> не должен содержать основной контент** — боковая колонка с рекламой внутри статьи
- **Тег <main> не может находиться внутри <article>** — главное содержимое вложено внутрь тега article

🔮 Шаг 08:

Миссия 9

📄 Сопоставьте ошибки и правильные решения при использовании тегов.

Ошибки:

Ответ:

- несколько <main>
- <aside> содержит основной текст статьи
- главный контент без семантики
- убрать тег article и обернуть всё в main

Правильные решения:

1. использовать один <main>
2. перенести текст из <aside> в <article>
3. оборачивать главный контент в <main>
4. разбить разные части на <article> или <aside>

Подсказка: Часто решение — не убрать теги, а распределить содержимое по правильным блокам.

- **несколько <main>** — использовать один <main>
- **<aside> содержит основной текст статьи** — перенести текст из <aside> в <article>
- **главный контент без семантики** — оборачивать главный контент в <main>
- **убрать тег article и обернуть всё в main** — разбить разные части на <article> или <aside>

🔮 Шаг 09:

Миссия 10

Расположите шаги при планировании семантики страницы с тегami main, article и aside:

1. Познакомиться с содержимым страницы (тексты, блоки)
2. Распределить части на главный, вспомогательный и независимые
3. Обернуть главный контент в <main>
4. Обернуть независимые части в <article> и вспомогательные в <aside>
5. Проверить вложение и корректность структуры

Подсказка: Начинайте с анализа содержания, а не сразу с тегов.

Ответ:

- Познакомиться с содержимым страницы (тексты, блоки)
- Распределить части на главный, вспомогательный и независимые
- Обернуть главный контент в <main>
- Обернуть независимые части в <article> и вспомогательные в <aside>
- Проверить вложение и корректность структуры

🔧 Шаг 10:

Миссия 11

У Лены три блока: навигация, основная статья, боковая панель. В каком порядке лучше расставить теги в HTML-структуре?

1. <aside>
2. <main>
3. <article>
4. внутри main сначала article, потом aside

Подсказка: main — это контейнер для основного, внутри которого может быть article + aside.

Ответ:

- <aside>
- <main>
- <article>
- внутри main сначала article, потом aside

🔧 Шаг 11:

Миссия 12

Выстроить порядок вложения тегов, чтобы структура была валидной:

1. <main>
2. <article>
3. <aside>
4. <header>

Подсказка: aside может быть внутри main, header обычно вне main.

Ответ:

- <main>
- <article>
- <aside>
- <header>

🔧 Шаг 12:

Миссия 13

Расставьте тройку этапов проверки семантики страницы перед сдачей:

1. Проверить, что только один <main>
2. **Убедиться, что данные независимые части** — <article> или <aside>
3. Убедиться, что <aside> не содержит основной контент

Подсказка: Начинайте с глобального, потом детали.

Ответ:

- Проверить, что только один <main>
- **Убедиться, что данные независимые части** — <article> или <aside>
- Убедиться, что <aside> не содержит основной контент

🔧 Шаг 13:

Миссия 14

Выстроить полный порядок вложения всех ключевых семантических блоков на странице:

1. <header><nav>
2. <main>
3. <article>
4. <aside>
5. <footer>

Подсказка: Основное содержимое скрыто внутри main, aside рядом, header и footer — на уровне страницы.

Ответ:

- <header>
- <nav>
- <main>
- <article>
- <aside>
- <footer>

Урок 2 - Грамотная структура документа

Шаг 01:

Миссия 2

Костя пишет страницу и хочет подключить метаданные (кодировка, viewport и т.д.). В каком теге их надо размещать?

Подсказка: Этот тег — контейнер для метаданных и заголовочного содержимого.

Ответ:

-
-
-

Шаг 02:

Миссия 3

Марина создает страницу с основным содержимым. Где нужно поместить видимые блоки текста и картинки?

Подсказка: Это раздел, который отображается пользователю.

Ответ:

-
-
-

Шаг 03:

Миссия 4

Игорь задаёт вопрос: может ли тег быть пустым, если у нас серверный рендер генерирует всё через скрипты?

Подсказка: Подумайте: браузер должен что-то показать, если скрипты не сработают.

Ответ:

- Да, это допустимо

- Нет, стоит добавить базовый контент

Шаг 04:

Миссия 6

 Сопоставьте элементы документа и их назначение.

Элементы:

Ответ:

- DOCTYPE
- head
- body
- title

Назначения:

1. обозначает тип и версию документа
2. контейнер для метаданных и связей
3. основная часть с содержимым страницы
4. имя страницы, отображаемое в браузере

Подсказка: Каждая часть HTML играет уникальную роль в структуре документа.

- **DOCTYPE** — обозначает тип и версию документа
- **head** — контейнер для метаданных и связей
- **body** — основная часть с содержимым страницы
- **title** — имя страницы, отображаемое в браузере

Шаг 05:

Миссия 7

 Сопоставьте тег и правильное положение в документе.

Теги:

Ответ:

- html
- head
- body
- meta charset

Позиции:

1. в самом верху после DOCTYPE
2. внутри , перед body
3. внутри , после head
4. внутри head, первым элементом

Подсказка: Обратите внимание на вложенность тегов.

- **html** — в самом верху после DOCTYPE
- **head** — внутри , перед body
- **body** — внутри , после head
- **meta charset** — внутри head, первым элементом

Шаг 06:

Миссия 8

 Сопоставьте ошибки структуры и их последствия.

Ошибка структуры:

Ответ:

- нет DOCTYPE
- мета внутри body
- title вне head
- body внутри head

Последствия:

1. браузер может работать в режиме quirks
2. несоответствие стандартам и неопределённый рендер
3. заголовок не отображается корректно
4. основной контент перестаёт работать

Подсказка: Ошибки вложения часто приводят к неожиданному поведению браузера.

- **нет DOCTYPE** — браузер может работать в режиме quirks
- **мета внутри body** — несоответствие стандартам и неопределённый рендер
- **title вне head** — заголовок не отображается корректно
- **body внутри head** — основной контент перестаёт работать

Шаг 07:

Миссия 9

 Сопоставьте элемент шаблона страницы и рекомендуемую часть документа.

Элемент страницы:

Ответ:

- панель навигации
- контент блога
- футер с контактами
- метаданные SEO

Документ:

1. внутри body, в header/nav
2. внутри body, в main/article
3. внутри body, в footer
4. внутри head

Подсказка: Где логичнее располагать навигацию, метаданные и содержимое?

- **панель навигации** — внутри body, в header/nav
- **контент блога** — внутри body, в main/article
- **футер с контактами** — внутри body, в footer
- **метаданные SEO** — внутри head

Шаг 08:

Миссия 11

Расположите в правильном порядке этапы создания базовой структуры HTML-документа:

1. Открыть тег
2. Добавить с метаданными
3. Добавить с содержимым
4. Закрыть тег

Подсказка: Начинайте с оболочки, потом — внутренние части, затем закрытие.

Ответ:

- Открыть тег
- Добавить с метаданными
- Добавить с содержимым
- Закрыть тег

Шаг 09:

Миссия 13

Антон создаёт страницу с навигацией, статьёй и футером. Какой порядок тегов внутри body будет логичен?

- 1.
- 2.
3. или
- 4.

Подсказка: Навигация часто вверху, футер — внизу.

Ответ:

-
-
- или
-

Шаг 10:

Миссия 14

Наташа уже создала , , . Теперь надо вставить: meta, title, script, содержимое. Какой логичный порядок?

1. meta charset
2. title
3. внешние ссылки и стили
4. контент в body
5. script внизу перед закрытием body

Подсказка: Скрипты часто подключают в конце, чтобы не блокировать загрузку.

Ответ:

- meta charset
- title
- внешние ссылки и стили
- контент в body
- script внизу перед закрытием body

🔔 Шаг 11:

Миссия 15

Вам нужно упорядочить шаги проверки структуры перед публикацией:

1. Проверить, что **DOCTYPE** — первая строка
2. Убедиться, что title внутри head
3. Проверить, что body содержит весь видимый контент
4. Убедиться, что теги закрыты корректно

Подсказка: Сначала глобальное, затем вложенное, затем закрытие.

Ответ:

- Проверить, что **DOCTYPE** — первая строка
- Убедиться, что title внутри head
- Проверить, что body содержит весь видимый контент
- Убедиться, что теги закрыты корректно

🔔 Шаг 12:

Миссия 16

Сергей создаёт страницу: html, head, body, nav, main, article, aside, footer. Расставьте в логическом порядке:

1. html
2. head
3. body
4. nav
5. main
6. article
7. aside
8. footer
9. закрывающие теги

Подсказка: body содержит другие семантические части, порядок изнутри наружу важен.

Ответ:

- html
- head
- body
- nav
- main
- article
- aside
- footer
- закрывающие теги

🔔 Шаг 13:

Миссия 17

Лена готовит шаблон страницы с полной структурой. Каким будет финальный порядок всех тегов?

1. DOCTYPE

2. html
3. head с meta, title, link
4. body
5. nav
6. main → article / section / aside
7. footer
8. закрыть body и html

Подсказка: Соберите всё воедино: глобальное — внутренняя семантика — закрытие.

Ответ:

- DOCTYPE
- html
- head с meta, title, link
- body
- nav
- main → article / section / aside
- footer
- закрыть body и html

Урок 3 - Атрибуты lang и dir

Шаг 01:

Миссия 1

Анна создаёт HTML-документ для сайта на русском языке. Какой атрибут она должна указать в теге ?

Подсказка: Этот атрибут сообщает, на каком языке написан документ.

Ответ:

- lang="ru"
- dir="rtl"
- lang="en"

Шаг 02:

Миссия 6

 Сопоставьте атрибут и его значение:

Ответ:

- lang
 - dir
 - lang + dir
 - отсутствие атрибутов
1. указывает язык документа или элемента
 2. указывает направление письма (ltr/rtl)
 3. указывает одновременно язык и направление
 4. браузер использует значения по умолчанию

Подсказка: Подумайте, что делает каждый атрибут отдельно и вместе.

- **lang** — указывает язык документа или элемента
- **dir** — указывает направление письма (ltr/rtl)

- **lang + dir** — указывает одновременно язык и направление
- **отсутствие атрибутов** — браузер использует значения по умолчанию

🔧 Шаг 03:

Миссия 7

📄 Сопоставьте ситуацию и правильную пару атрибутов в HTML:

Ответ:

- страница на арабском
- блок на английском в русском документе
- страница на русском
- страница без указания языка

1. lang="ar" dir="rtl"
2. lang="en"
3. lang="ru"
4. (нет lang, нет dir)

Подсказка: Помните: direction нужен только если это справа-налево.

- **страница на арабском** — lang="ar" dir="rtl"
- **блок на английском в русском документе** — lang="en"
- **страница на русском** — lang="ru"
- **страница без указания языка** — (нет lang, нет dir)

🔧 Шаг 04:

Миссия 8

📄 Сопоставьте ошибку использования атрибутов и её следствие:

Ответ:

- lang неверный код
- dir не соответствует языку
- отсутствует lang на вложенном элементе
- dir у внутреннего элемента при глобальном ltr

1. экранные читалки неправильно озвучат язык
2. текст отображается в неверном направлении
3. вложенный текст остаётся на основном языке
4. несоответствие направления и языка внутри

Подсказка: Подумайте о воздействии на доступность и парсеры.

- **lang неверный код** — экранные читалки неправильно озвучат язык
- **dir не соответствует языку** — текст отображается в неверном направлении
- **отсутствует lang на вложенном элементе** — вложенный текст остаётся на основном языке
- **dir у внутреннего элемента при глобальном ltr** — несоответствие направления и языка внутри

🔧 Шаг 05:

Миссия 10

Расположите шаги при добавлении многоязычности на страницу:

1. Указать lang и dir на <html>
2. Определить участки текста другого языка
3. Проставить локальный lang (и возможно dir) на элементы
4. Проверить корректность направления и языка

Подсказка: Сначала задайте глобальные атрибуты, потом уточните локально.

Ответ:

- Указать lang и dir на
- Определить участки текста другого языка
- Проставить локальный lang (и возможно dir) на элементы
- Проверить корректность направления и языка

Шаг 06:

Миссия 11

Виктор уже указал lang и dir на . Что дальше?

1. Вставить локальные lang для блоков другого языка
2. Указать dir, если направление текста отличается
3. Проверить, чтобы не было двух conflicting dir
4. Проверить рендер и озвучивание

Подсказка: Локальные атрибуты уточняют глобальные, но не конфликтуют.

Ответ:

- Вставить локальные lang для блоков другого языка
- Указать dir, если направление текста отличается
- Проверить, чтобы не было двух conflicting dir
- Проверить рендер и озвучивание

Шаг 07:

Миссия 12

Ольга готовит страницу с арабскими и английскими фразами. Какой порядок применения атрибутов?

1. lang="ar" dir="rtl" на
2. **английский блок** — lang="en" внутри
3. при необходимости dir="ltr" на английском блоке
4. проверка, чтобы основной текст правильно бегнул

Подсказка: Глобальное направление устанавливается первым, потом уточнение локально.

Ответ:

- lang="ar" dir="rtl" на
- **английский блок** — lang="en" внутри
- при необходимости dir="ltr" на английском блоке
- проверка, чтобы основной текст правильно вернулся

Шаг 08:

Миссия 13

Женя делает страницу с основным русским текстом и вставками арабского. Какой порядок?

1. lang="ru" на
2. определить вставки арабского
3. **на вставках** — lang="ar" dir="rtl"
4. проверить отображение и правильность направления

Подсказка: Уточнения внутри локальных элементов.

Ответ:

- lang="ru" на
- определить вставки арабского
- **на вставках** — lang="ar" dir="rtl"
- проверить отображение и правильность направления

Шаг 09:

Миссия 14

У Саши уже есть . Какие шаги проверки структуры атрибутов?

1. Убедиться, что dir соответствует lang
2. Проверить вложенные блоки с другим языком
3. Проверить, нет ли противоречивых dir внутри
4. Проверить поведение текста при смене языка

Подсказка: Проверяйте согласованность языка и направления.

Ответ:

- Убедиться, что dir соответствует lang
- Проверить вложенные блоки с другим языком
- Проверить, нет ли противоречивых dir внутри
- Проверить поведение текста при смене языка

Шаг 10:

Миссия 15

Вика добавляет блок цитаты на арабском внутрь русского документа. Какой порядок действий?

1. Выделить тег цитаты (blockquote или p)
2. Проставить lang="ar" dir="rtl" для этого блока
3. Оставить общий lang="ru" dir="ltr" в
4. Проверить визуально и при чтении экраном

Подсказка: Локальный блок — сначала задать язык, затем направление.

Ответ:

- Выделить тег цитаты (blockquote или p)
- Проставить lang="ar" dir="rtl" для этого блока
- Оставить общий lang="ru" dir="ltr" в
- Проверить визуально и при чтении экраном

Шаг 11:

Миссия 16

Кирилл делает сайт на арабском языке, но часть — на английском. Расположите порядок:

1. Указать `lang="ar" dir="rtl"` глобально
2. Найти секции на английском
3. Проставить `lang="en"` (и `dir="ltr"`, если нужно) там
4. Проверить отсутствие конфликтов и визуализацию

Подсказка: Глобальные установки → локальные уточнения → проверка.

Ответ:

- Указать `lang="ar" dir="rtl"` глобально
- Найти секции на английском
- Проставить `lang="en"` (и `dir="ltr"`, если нужно) там
- Проверить отсутствие конфликтов и визуализацию

Шаг 12:

Миссия 17

Алина готовит многоязычный шаблон. Упорядочьте все шаги:

1. Указать `lang + dir` на
2. Определить все участки на других языках
3. Проставить локальные `lang` (и `dir`, если направление отличается)
4. Проверить корректность отображения и доступности

Подсказка: Структура: глобальное → локальное → проверка.

Ответ:

- Указать `lang + dir` на
- Определить все участки на других языках
- Проставить локальные `lang` (и `dir`, если направление отличается)
- Проверить корректность отображения и доступности

Урок 4 - Валидация HTML-кода

Шаг 01:

Миссия 1

Юля сверстала главную страницу сайта, но забыла проверить HTML-код. Что стоит сделать в первую очередь?

Подсказка: Есть специальные онлайн-инструменты от W3C.

Ответ:

- Проверить HTML через валидатор W3C
- Открыть файл только в браузере
- Запустить антивирусную проверку
- Очистить кэш браузера

Шаг 02:

Миссия 2

Марк проверил страницу через валидатор, и получил предупреждение про отсутствие `lang` в теге `<html>`. Что это означает?

Подсказка: Этот атрибут помогает браузерам и вспомогательным технологиям.

Ответ:

- **Не указан язык документа, что влияет на доступность**
- Неверный DOCTYPE
- Не хватает стилей CSS

Шаг 03:

Миссия 3

Света использует тег `<div>` внутри `<ul>`. Валидатор выдаёт ошибку. Почему?

Подсказка: Некоторые теги можно использовать только в определённых структурах.

Ответ:

- **`<ul>` может содержать только `<li>`**
- `<div>` не должен быть вложенным
- `<ul>` нельзя использовать внутри `<body>`
- **`<div>` — устаревший тег**

Шаг 04:

Миссия 4

Дизайнер Артём случайно использовал несуществующий тег `<box>`. Что произойдёт при валидации?

Подсказка: Все теги должны быть частью стандарта.

Ответ:

- **Валидатор укажет на неизвестный тег**
- Просто проигнорируется без ошибок
- Автоматически заменится на `<div>`

Шаг 05:

Миссия 5

Frontend-разработчица Кира не закрыла тег `<img>`. Что скажет валидатор?

Подсказка: Не все теги требуют закрывающего парного тега, но синтаксис должен быть точен.

Ответ:

- **Ошибка: тег `<img>` должен быть самозакрывающимся или корректно оформлен**
- Браузер автоматически добавит alt
- `<img>` обязательно требует парный тег
- Ошибки не будет

Шаг 06:

Миссия 7

 Соотнесите тип сообщения валидатора и его значение:

Ответ:

- Error
- Warning
- Info
- Hint

1. Нарушение стандарта, нужно исправить

2. Незначительное, но рекомендуемое исправление
3. Дополнительная информация по коду
4. Подсказка к улучшению кода

Подсказка: Ошибки мешают валидному HTML; предупреждения — потенциальные проблемы.

- **Error** — Нарушение стандарта, нужно исправить
- **Warning** — Незначительное, но рекомендуемое исправление
- **Info** — Дополнительная информация по коду
- **Hint** — Подсказка к улучшению кода

Шаг 07:

Миссия 9

 Соотнесите сообщения валидатора и возможные действия разработчика:

Ответ:

- Unknown element <xyz>
 - Element “p” not allowed here;
 - Attribute “foo” not allowed on element “div”;
 - Missing “alt” attribute on ;
1. Удалить или заменить тег
 2. Изменить структуру / переместить тег
 3. Удалить атрибут или использовать другой тег
 4. Добавить атрибут alt с текстом

Подсказка: Валидатор указывает, что не разрешено, а вы решаете, как это поправить.

- **Unknown element** — Удалить или заменить тег
- **Element “p” not allowed here;** — Изменить структуру / переместить тег
- **Attribute “foo” not allowed on element “div”;** — Удалить атрибут или использовать другой тег
- **Missing “alt” attribute on ;** — Добавить атрибут alt с текстом

Шаг 08:

Миссия 10

 Соотнесите валидатор и формат кода, который он проверяет:

Ответ:

- W3C HTML Validator
 - W3C CSS Validator
 - Анализатор Accessibility
 - Локальный линтер (например, HTMLHint)
1. валидность HTML-документа
 2. валидность CSS-кода
 3. доступность и семантика
 4. статический анализ кода при разработке

Подсказка: Разные инструменты проверяют разные аспекты веб-страницы.

- **W3C HTML Validator** — валидность HTML-документа
- **W3C CSS Validator** — валидность CSS-кода

- **Анализатор Accessibility** — доступность и семантика
- **Локальный линтер (например, HTMLHint)** — статический анализ кода при разработке

🔮 Шаг 09:

Миссия 11

📖 Соотнесите подход к исправлению и момент, когда его применяют:

Ответ:

- Исправление на этапе разработки
- Исправление перед деплоем (на CI)
- Исправление по отчёту после публикации
- Профилактическая проверка перед слиянием (merge)

1. при написании кода локально
2. на стадии автоматического тестирования
3. когда пользователи сообщают об ошибках
4. в процессе code review / pre-merge

Подсказка: Ошибки можно ловить на разных этапах life-cycle.

- **Исправление на этапе разработки** — при написании кода локально
- **Исправление перед деплоем (на CI)** — на стадии автоматического тестирования
- **Исправление по отчёту после публикации** — когда пользователи сообщают об ошибках
- **Профилактическая проверка перед слиянием (merge)** — в процессе code review / pre-merge

🔮 Шаг 10:

Миссия 12

Расположите шаги валидации страницы перед публикацией:

1. Локальная проверка кода (линтеры)
2. Проверка через онлайн-валидатор HTML/CSS
3. Тестирование на браузерах и устройствах
4. Проверка доступности и семантики
5. Релиз страницы

Подсказка: Начинайте с простого синтаксиса, затем проверку визуала и доступность.

Ответ:

- Локальная проверка кода (линтеры)
- Проверка через онлайн-валидатор HTML/CSS
- Тестирование на браузерах и устройствах
- Проверка доступности и семантики
- Релиз страницы

🔮 Шаг 11:

Миссия 13

Лена уже проверила HTML валидатором. Что логично делать дальше?

1. Исправить все ошибки, отмеченные валидатором
2. Проверить страницу на разных браузерах

3. Запустить линтер на CSS и JS
4. Вновь прогнать валидатор после исправлений
5. Выпустить страницу

Подсказка: После исправления кода важно повторно проверить.

Ответ:

- Исправить все ошибки, отмеченные валидатором
- Проверить страницу на разных браузерах
- Запустить линтер на CSS и JS
- Вновь прогнать валидатор после исправлений
- Выпустить страницу

Шаг 12:

Миссия 14

Вадим работает с шаблоном, где несколько страниц генерируются автоматически. Какой порядок валидации применить к шаблонам?

1. Проверка шаблона вручную на сайте разработки
2. Валидация HTML-кода шаблона
3. Запуск автоматического линта при сборке
4. Проверка конечных страниц на валидность
5. Публикация

Подсказка: Проверка должна быть на каждом этапе генерации.

Ответ:

- Проверка шаблона вручную на сайте разработки
- Валидация HTML-кода шаблона
- Запуск автоматического линта при сборке
- Проверка конечных страниц на валидность
- Публикация

Шаг 13:

Миссия 15

Дима перед мёрж-реквестом хочет убедиться в валидности HTML всего проекта. В каком порядке он должен действовать?

1. Запустить линтер на все файлы
2. Прогнать валидатор на тематических страницах
3. Исправить найденные ошибки
4. Повторно проверить после исправлений
5. Слить изменения в основную ветку

Подсказка: Проверка — исправление — перепроверка — мёрж.

Ответ:

- Запустить линтер на все файлы
- Прогнать валидатор на тематических страницах
- Исправить найденные ошибки
- Повторно проверить после исправлений
- Слить изменения в основную ветку

Шаг 14:

Миссия 16

София готовит многостраничный сайт: основные шаги валидации всех страниц?

1. Сбор всех HTML-страниц
2. Запустить валидатор на каждую страницу
3. Сгруппировать ошибки по типам и приоритетам
4. Исправить сначала критические, затем предупреждения
5. Прогнать валидатор ещё раз и подтвердить чистоту кода

Подсказка: Обработка ошибок по приоритету — важная часть.

Ответ:

- Сбор всех HTML-страниц
- Запустить валидатор на каждую страницу
- Сгруппировать ошибки по типам и приоритетам
- Исправить сначала критические, затем предупреждения
- Прогнать валидатор ещё раз и подтвердить чистоту кода

Модуль 2 - ✂ Продвинутое формы

Урок 1 - Новые input-типы date, color, range

Шаг 01:

Миссия 1

Оля делает форму заказа, и хочет пользователь мог выбрать дату доставки. Какой тип input ей использовать?

Подсказка: Тип для выбора календарной даты.

Ответ:

- **type="date"**
- type="text"
- type="range"

Шаг 02:

Миссия 2

Игорь хочет, чтобы пользователь мог выбрать цвет для фона профиля. Какой input-type он применит?

Подсказка: Тип, который открывает палитру цветов.

Ответ:

- **type="color"**
- type="checkbox"
- type="text"

Шаг 03:

Миссия 3

Катя рисует ползунок уровня громкости. Какой input-type подойдет?

Подсказка: Ползунок — это диапазон значений.

Ответ:

- **type="range"**
- type="number"
- type="slider"

Шаг 04:

Миссия 4

Пётр хочет, чтобы форма ввода даты показывала календарь и не позволяла выбрать прошлые даты. Какого атрибута не хватает?

Подсказка: Это атрибут, ограничивающий минимальную дату.

Ответ:

- **min="YYYY-MM-DD"**
- max="YYYY-MM-DD"
- step="1"

Шаг 05:

Миссия 5

Лена хочет, чтобы ползунок устанавливал шаг в 5 единиц. Какой атрибут ей нужен?

Подсказка: Ползунок может меняться не с любой дробью, а с шагом.

Ответ:

- **step="5"**
- interval="5"
- increment="5"

Шаг 06:

Миссия 6

 Сопоставьте каждую задачу с подходящим input-типом:

Ответ:

- Выбор даты рождения
- Выбор цвета униформы
- Настройка громкости в приложении
- Выбор минимальной даты события в календаре

1. date
2. color
3. range
4. date + атрибут min

Подсказка: Подумайте, что именно должен выбрать пользователь и какие ограничения нужны.

- **Выбор даты рождения** — date
- **Выбор цвета униформы** — color
- **Настройка громкости в приложении** — range
- **Выбор минимальной даты события в календаре** — date + атрибут min

Шаг 07:

Миссия 7

 Сопоставьте атрибуты input range и что они делают:

Ответ:

- min
- max
- step
- value

1. минимальное значение ползунка
2. максимальное значение ползунка
3. шаг изменения
4. начальное значение

Подсказка: Ползунок range имеет 4 основных атрибута контроля.

- **min** — минимальное значение ползунка
- **max** — максимальное значение ползунка
- **step** — шаг изменения
- **value** — начальное значение

 **Шаг 08:**

Миссия 8

 Сопоставьте примеры использования и возможные ошибки:

Ответ:

- `<input type="date" min="2025-01-01" />`
- `<input type="color" value="blue" />`
- `<input type="range" step="0.5" max="10" />`
- `<input type="date" max="2020-01-01" />`

1. корректно, min ограничивает прошлое
2. ошибка, значение цвета не в формате hex
3. корректно, дробный шаг допустим
4. ошибка, max меньше сегодняшней даты (если нужно будущие даты)

Подсказка: Форматы данных имеют строгие формы, цвет — hex-код или нечто подобное.

- — корректно, min ограничивает прошлое
- — ошибка, значение цвета не в формате hex
- — корректно, дробный шаг допустим
- — ошибка, max меньше сегодняшней даты (если нужно будущие даты)

 **Шаг 09:**

Миссия 9

 Сопоставьте input-type и формат отображения значения:

Ответ:

- date
- color
- range
- color с value по умолчанию

1. пользователь видит календарь

2. появится палитра цвета
3. ползунок
4. предварительный цвет выбранный

Подсказка: Как визуально реагирует UI-элемент на value?

- **date** — пользователь видит календарь
- **color** — появится палитра цвета
- **range** — ползунок
- **color с value по умолчанию** — предварительный цвет выбранный

Шаг 10:

Миссия 10

 Соотнесите варианты формы с тем, как они обрабатываются при отправке:

Ответ:

- `<input type="date" name="birthday" />`
- `<input type="color" name="themeColor" />`
- `<input type="range" name="volume" value="3" />`
- `<input type="range" name="opacity" min="0" max="1" step="0.1" />`

1. строка даты в формате YYYY-MM-DD
2. HEX-код цвета, например #rrggbb
3. число в диапазоне по умолчанию 0-100 если не указан min/max
4. число с дробной частью

Подсказка: У разных типов разные форматы данных при отправке формы.

- — строка даты в формате YYYY-MM-DD
- — HEX-код цвета, например #rrggbb
- — число в диапазоне по умолчанию 0-100 если не указан min/max
- — число с дробной частью

Шаг 11:

Миссия 11

Расположите шаги по добавлению input-type="date" с ограничениями в форме бронирования:

1. Определить допустимые даты
2. Установить атрибут min и max
3. Добавить input type="date"
4. Проверить, что выбор вне диапазона не разрешён

Подсказка: Сначала про ограничения, потом элемент, потом проверка.

Ответ:

- Определить допустимые даты
- Установить атрибут min и max
- Добавить input type="date"
- Проверить, что выбор вне диапазона не разрешён

Шаг 12:

Миссия 12

Настя делает форму настроек: сначала цвет, потом диапазон, затем дату. Какой порядок кода логически правильный?

1. `<label>Theme Color</label> <input type="color" ... />`
2. `<label>Volume</label> <input type="range" ... />`
3. `<label>Delivery Date</label> <input type="date" ... />`
4. `<button>Submit</button>`

Подсказка: Обычно задаются сначала визуально влияющие, потом функциональные элементы.

Ответ:

- Theme Color
- Volume
- Delivery Date
- Submit

Шаг 13:

Миссия 13

Алекс настроил input range: min, max, step. Он хочет убедиться, что форма работает. Какой порядок проверки?

1. Написать HTML с input type="range" и атрибутами
2. Задать начальное value внутри диапазона
3. Проверить визуальное поведение ползунка в браузере
4. Попытаться ввести значение вне диапазона (если возможно через developer tools или вручную)
5. Убедиться, что отправка формы возвращает ожидаемое значение

Подсказка: После написания кода важно тестировать границы и отpravку.

Ответ:

- Написать HTML с input type="range" и атрибутами
- Задать начальное value внутри диапазона
- Проверить визуальное поведение ползунка в браузере
- Попытаться ввести значение вне диапазона (если возможно через developer tools или вручную)
- Убедиться, что отправка формы возвращает ожидаемое значение

Шаг 14:

Миссия 14

Вася делает настройку формы с датой и диапазоном громкости. Упорядочите действия:

1. Добавить input type="range" для громкости
2. Добавить input type="date" для планируемой даты события
3. Установить min/max/step для диапазона громкости
4. Убедиться, что date не может быть в прошлом
5. Тестировать форму

Подсказка: Сначала вставляют поля, потом задают ограничения, потом проверяют.

Ответ:

- Добавить input type="range" для громкости
- Добавить input type="date" для планируемой даты события

- Установить min/max/step для диапазона громкости
- Убедиться, что date не может быть в прошлом
- Тестировать форму

🔧 Шаг 15:

Миссия 15

Наташа хочет сделать форму с предустановленным цветом и диапазоном настроек. Какой порядок действий будет последовательным?

1. Добавить input type="color" с value по умолчанию
2. Добавить input type="range" с min/max/step
3. Добавить input type="date" с ограничением min
4. Добавить кнопку submit
5. Проверить все элементы в UI и их значения

Подсказка: Сначала элементы, затем ограничения, затем проверка интерфейса.

Ответ:

- Добавить input type="color" с value по умолчанию
- Добавить input type="range" с min/max/step
- Добавить input type="date" с ограничением min
- Добавить кнопку submit
- Проверить все элементы в UI и их значения

🔧 Шаг 16:

Миссия 16

Все готово! Какой полный порядок создания формы с date, color и range, с проверкой и публикацией?

1. Проектирование: решить, какие поля нужны
2. Написать HTML с полями (date, color, range)
3. Установить атрибуты min, max, step, value, ограничения даты
4. Протестировать поля в разных браузерах / устройствах
5. Добавить форму на сайт / опубликовать изменения

Подсказка: От идеи до публикации: сначала дизайн → код → проверка → релиз.

Ответ:

- Проектирование: решить, какие поля нужны
- Написать HTML с полями (date, color, range)
- Установить атрибуты min, max, step, value, ограничения даты
- Протестировать поля в разных браузерах / устройствах
- Добавить форму на сайт / опубликовать изменения

📖 Урок 2 - Валидация required, pattern, minmax

🔧 Шаг 01:

Миссия 1

Программист Макс создает форму регистрации и хочет, чтобы пользователь обязательно указал имя. Какой атрибут HTML нужно использовать для обязательного поля?

Подсказка: Подумайте, как заставить браузер не пропускать пустое поле.

Ответ:

- **required**
- pattern
- min
- max

🔮 **Шаг 02:**

Миссия 2

Аня хочет, чтобы в поле ввода номера телефона можно было вводить только цифры и дефисы. Какой атрибут поможет ограничить формат ввода?

Подсказка: Регулярные выражения в HTML помогут задать правила для ввода.

Ответ:

- **pattern**
- required
- min
- max

🔮 **Шаг 03:**

Миссия 3

Иван создает форму для заказа товара, где нужно ограничить количество товаров от 1 до 10. Какие атрибуты помогут это реализовать?

Подсказка: Обратите внимание на минимальное и максимальные значения.

Ответ:

- **min**
- **max**
- pattern
- required

🔮 **Шаг 04:**

Миссия 4

Катя разрабатывает форму, где пользователь вводит почту. Какой атрибут HTML поможет убедиться, что поле не осталось пустым?

Подсказка: Этот атрибут блокирует отправку формы, если поле пустое.

Ответ:

- **required**
- pattern
- min
- max

🔮 **Шаг 05:**

Миссия 5

Максим хочет разрешить ввод только латинских букв в поле для имени пользователя. Какой атрибут поможет задать такое ограничение?

Подсказка: Используйте регулярное выражение для указания допустимых символов.

Ответ:

- **pattern**="[^][A-Za-z]+^{\$}"
- required
- min
- max

Шаг 06:

Миссия 6

Сопоставьте атрибуты формы с их назначением.

Ответ:

- required
- pattern
- min
- max

1. Определяет минимально допустимое значение для числового поля.
2. Обязательное заполнение поля.
3. Устанавливает максимальное значение для числового поля.
4. Задаёт регулярное выражение для проверки формата ввода.

Подсказка: Подумайте, какой атрибут связан с проверкой формата, а какой с обязательностью.

- **required** — Обязательное заполнение поля.
- **pattern** — Задаёт регулярное выражение для проверки формата ввода.
- **min** — Определяет минимально допустимое значение для числового поля.
- **max** — Устанавливает максимальное значение для числового поля.

Шаг 07:

Миссия 7

Сопоставьте возможные ошибки ввода с атрибутом, который помогает их предотвратить.

Ответ:

- Пользователь оставил поле пустым
- Введён недопустимый формат
- Значение меньше минимального
- Значение больше максимального

1. required
2. pattern
3. min
4. max

Подсказка: Вспомните, какой атрибут отвечает за обязательность, а какой – за ограничение по числам.

- **Пользователь оставил поле пустым** — required
- **Введён недопустимый формат** — pattern
- **Значение меньше минимального** — min
- **Значение больше максимального** — max

Шаг 08:

Миссия 8

Сопоставьте ошибки пользователя с текстом подсказки, который поможет их избежать.

Ответ:

- Пропуск обязательного поля
- Ввод неправильного формата телефона
- Ввод числа меньше допустимого
- Ввод числа больше допустимого

1. Пожалуйста, заполните это поле.
2. Введите номер в формате +7-999-999-99-99.
3. Значение не может быть меньше 1.
4. Значение не может превышать 100.

Подсказка: Тексты подсказок должны быть понятны и прямо указывать на проблему.

- **Пропуск обязательного поля** — Пожалуйста, заполните это поле.
- **Ввод неправильного формата телефона** — Введите номер в формате +7-999-999-99-99.
- **Ввод числа меньше допустимого** — Значение не может быть меньше 1.
- **Ввод числа больше допустимого** — Значение не может превышать 100.

Шаг 09:

Миссия 9

Сопоставьте типы полей формы с атрибутами валидации, которые для них наиболее подходят.

Ответ:

- email
 - tel
 - number
 - text
1. required и pattern для формата почты
 2. pattern для номера телефона
 3. min и max для диапазона чисел
 4. required и pattern для имени пользователя

Подсказка: Разные типы полей требуют разных правил валидации.

- **email** — required и pattern для формата почты
- **tel** — pattern для номера телефона
- **number** — min и max для диапазона чисел
- **text** — required и pattern для имени пользователя

Шаг 10:

Миссия 10

Сопоставьте атрибуты HTML с типом проверки, которую они обеспечивают.

Ответ:

- required
- pattern

- min
- max

1. Обязательное заполнение поля
2. Проверка на соответствие шаблону
3. Минимальное число
4. Максимальное число

Подсказка: Каждый атрибут отвечает за свой тип ограничения.

- **required** — Обязательное заполнение поля
- **pattern** — Проверка на соответствие шаблону
- **min** — Минимальное число
- **max** — Максимальное число

Шаг 11:

Миссия 11

Расположите этапы проверки формы с атрибутами required, pattern, min, max в правильном порядке.

1. Проверка, что обязательные поля заполнены (required)
2. Проверка формата ввода через регулярное выражение (pattern)
3. Проверка, что значение не меньше минимального (min)
4. Проверка, что значение не превышает максимального (max)

Подсказка: Сначала проверяем наличие данных, потом их формат, затем числовые ограничения.

Ответ:

- Проверка, что обязательные поля заполнены (required)
- Проверка формата ввода через регулярное выражение (pattern)
- Проверка, что значение не меньше минимального (min)
- Проверка, что значение не превышает максимального (max)

Шаг 12:

Миссия 12

Упорядочьте действия, которые выполняет браузер при валидации формы с полями, использующими required, pattern, min и max.

1. Проверяет, что обязательные поля не пусты (required)
2. Сравнивает введённый текст с заданным регулярным выражением (pattern)
3. Проверяет, что числовое значение не меньше min
4. Проверяет, что числовое значение не больше max
5. Отправляет форму, если все проверки пройдены

Подсказка: Последовательность проверок важна для корректной работы формы.

Ответ:

- Проверяет, что обязательные поля не пусты (required)
- Сравнивает введённый текст с заданным регулярным выражением (pattern)
- Проверяет, что числовое значение не меньше min
- Проверяет, что числовое значение не больше max
- Отправляет форму, если все проверки пройдены

Шаг 13:

Миссия 13

Упорядочите шаги создания валидации поля ввода с ограничением формата и обязательностью.

1. Добавить атрибут required
2. Задать регулярное выражение через pattern
3. Написать пользовательское сообщение об ошибке
4. Проверить форму в браузере

Подсказка: Сначала настройте логику, потом протестируйте её.

Ответ:

- Добавить атрибут required
- Задать регулярное выражение через pattern
- Написать пользовательское сообщение об ошибке
- Проверить форму в браузере

Шаг 14:

Миссия 14

Упорядочите этапы создания поля ввода с числовыми ограничениями от 5 до 20 и обязательным заполнением.

1. Добавить атрибут required
2. Задать min=5
3. Задать max=20
4. Проверить, что форма не отправляется при нарушении ограничений

Подсказка: Помните, что сначала указываются условия, затем проводится тестирование.

Ответ:

- Добавить атрибут required
- Задать min=5
- Задать max=20
- Проверить, что форма не отправляется при нарушении ограничений

Урок 3 - Автозаполнение и плейсхолдеры

Шаг 01:

Миссия 1

Алексей создает форму регистрации и хочет, чтобы браузер помог пользователям автоматически заполнять поля. Какой атрибут HTML отвечает за автозаполнение?

Подсказка: Автозаполнение связано с подсказками браузера о ранее введенных данных.

Ответ:

- **autocomplete**
- placeholder
- required
- pattern

Шаг 02:

Миссия 2

Оля создает форму с полем для ввода имени пользователя. Какой placeholder она должна указать, чтобы пользователю было понятно, что нужно вводить?

Подсказка: Плейсхолдер — это пример или подсказка внутри поля.

Ответ:

- **Введите ваше имя пользователя**
- Обязательное поле
- 123456

🔗 Шаг 03:

Миссия 3

Игорь хочет, чтобы браузер не сохранял данные поля с паролем для безопасности. Какой атрибут он должен использовать?

Подсказка: Этот атрибут управляет поведением автозаполнения.

Ответ:

- **autocomplete="off"**
- placeholder="Пароль"
- required

🔗 Шаг 04:

Миссия 4

Катя создала форму с полем email и добавила placeholder «example@mail.com». Что она хочет показать пользователям?

Подсказка: Подумайте, какую роль играет плейсхолдер в восприятии поля.

Ответ:

- **Пример правильного формата email**
- Требование заполнить поле
- Подсказка об ошибке

🔗 Шаг 05:

Миссия 5

Сопоставьте атрибуты с их описанием и назначением в форме с автозаполнением и плейсхолдерами.

Ответ:

- autocomplete
 - placeholder
 - readonly
 - required
1. Подсказка, показывающая пример значения внутри поля
 2. Позволяет браузеру автоматически подставлять данные пользователя
 3. Запрещает редактирование поля
 4. Обязательное для заполнения поле

Подсказка: Каждый атрибут управляет разным аспектом поведения формы.

- **autocomplete** — Позволяет браузеру автоматически подставлять данные пользователя
- **placeholder** — Подсказка, показывающая пример значения внутри поля

- **readonly** — Запрещает редактирование поля
- **required** — Обязательное для заполнения поле

Шаг 06:

Миссия 6

Сопоставьте типы автозаполнения с подходящими значениями атрибута autocomplete для формы профиля.

Ответ:

- Имя пользователя
- Пароль
- Email
- Адрес доставки

1. username
2. current-password
3. email
4. shipping street-address

Подсказка: Значения autocomplete помогают браузеру распознавать тип данных.

- **Имя пользователя** — username
- **Пароль** — current-password
- **Email** — email
- **Адрес доставки** — shipping street-address

Шаг 07:

Миссия 7

Сопоставьте описания полей формы с лучшими значениями autocomplete для автозаполнения.

Ответ:

- Поле для ввода телефона
- Поле для ввода имени
- Поле для ввода фамилии
- Поле для ввода даты рождения

1. tel
2. given-name
3. family-name
4. bday

Подсказка: Помните, что для разных данных есть стандартизированные значения autocomplete.

- **Поле для ввода телефона** — tel
- **Поле для ввода имени** — given-name
- **Поле для ввода фамилии** — family-name
- **Поле для ввода даты рождения** — bday

Шаг 08:

Миссия 8

Сопоставьте примеры placeholder'ов с полями формы, чтобы они были максимально полезны и понятны.

Ответ:

- Поле ввода email
- Поле ввода номера телефона
- Поле ввода имени
- Поле ввода пароля

1. example@mail.com
2. +7 (999) 999-99-99
3. Иван
4. Минимум 8 символов

Подсказка: Хороший плейсхолдер — это короткая подсказка с примером или требованием.

- **Поле ввода email** — example@mail.com
- **Поле ввода номера телефона** — +7 (999) 999-99-99
- **Поле ввода имени** — Иван
- **Поле ввода пароля** — Минимум 8 символов

 Шаг 09:

Миссия 9

Расположите правильную последовательность действий при настройке поля с автозаполнением и плейсхолдером для email.

1. Добавить атрибут placeholder с примером email
2. Задать атрибут autocomplete со значением email
3. Проверить, что форма корректно работает и браузер предлагает автозаполнение
4. Добавить тип input="email" для проверки формата

Подсказка: Сперва опишите внешний вид, потом добавьте функциональность и протестируйте.

Ответ:

- Добавить атрибут placeholder с примером email
- Задать атрибут autocomplete со значением email
- Проверить, что форма корректно работает и браузер предлагает автозаполнение
- Добавить тип input="email" для проверки формата

 Шаг 10:

Миссия 10

Упорядочьте этапы создания поля ввода номера телефона с автозаполнением и подсказкой.

1. Добавить атрибут type="tel"
2. Добавить placeholder с примером формата номера
3. Задать autocomplete="tel" для автозаполнения
4. Проверить работу автозаполнения и форматирования

Подсказка: Помните, что сначала задаются тип и внешний вид, затем функциональность и тестирование.

Ответ:

- Добавить атрибут type="tel"
- Добавить placeholder с примером формата номера
- Задать autocomplete="tel" для автозаполнения

- Проверить работу автозаполнения и форматирования

Шаг 11:

Миссия 11

Расположите шаги при создании формы с обязательным полем имени пользователя с автозаполнением и плейсхолдером.

1. Добавить атрибут `required`
2. Задать `placeholder` с примером имени
3. Задать `autocomplete="username"`
4. Проверить форму на корректность автозаполнения

Подсказка: Не забудьте сначала указать обязательность, потом подсказку, потом автозаполнение.

Ответ:

- Добавить атрибут `required`
- Задать `placeholder` с примером имени
- Задать `autocomplete="username"`
- Проверить форму на корректность автозаполнения

Шаг 12:

Миссия 12

Расположите действия по настройке поля для пароля с отключённым автозаполнением и плейсхолдером.

1. Добавить атрибут `type="password"`
2. Добавить `placeholder` с подсказкой о минимальной длине
3. Добавить `autocomplete="off"` для безопасности
4. Проверить, что браузер не предлагает автозаполнение

Подсказка: Важно обеспечить безопасность поля пароля.

Ответ:

- Добавить атрибут `type="password"`
- Добавить `placeholder` с подсказкой о минимальной длине
- Добавить `autocomplete="off"` для безопасности
- Проверить, что браузер не предлагает автозаполнение

Урок 4 - Группировка `fieldset` и `legend`

Шаг 01:

Миссия 1

Разработчик Ирина создаёт форму заказа с несколькими связанными полями и хочет визуально их сгруппировать. Какой тег HTML ей лучше использовать?

Подсказка: Подумайте, какой тег помогает логически и визуально объединить элементы формы.

Ответ:

- **`fieldset`**
- `div`
- `section`
- `article`

Шаг 02:

Миссия 2

Виталий добавляет заголовок для группы полей в форме, чтобы пользователи понимали, что они заполняют. Какой тег следует использовать для этого заголовка?

Подсказка: Этот тег всегда идет внутри fieldset и описывает группу.

Ответ:

- **legend**
- h1
- label

Шаг 03:

Миссия 3

Марина хочет убедиться, что у поля ввода внутри fieldset всегда есть подпись, чтобы пользователь понимал, что вводить. Что нужно использовать?

Подсказка: Подпись должна быть связана с конкретным input.

Ответ:

- **label**
- legend
- span
- fieldset

Шаг 04:

Миссия 4

Алексей создает форму с несколькими группами полей, но забыл использовать fieldset . Какие проблемы могут возникнуть?

Подсказка: Подумайте о восприятии формы пользователями и доступности.

Ответ:

- **Пользователям будет сложнее понять структуру формы**
- Форма не будет отправляться
- Форматирование HTML сломается

Шаг 05:

Миссия 5

 Задача: Сопоставьте элементы с их описаниями.

Элементы:

Ответ:

- fieldset
- legend
- label
- input

Описание:

1. Определяет заголовок группы элементов формы.
2. Связывает текст с полем ввода.
3. Обозначает отдельную группу полей.

4. Элемент для ввода данных пользователем.

Подсказка:

Обратите внимание на роли каждого элемента в структуре формы.

- **fieldset** — Обозначает отдельную группу полей.
- **legend** — Определяет заголовок группы элементов формы.
- **label** — Связывает текст с полем ввода.
- **input** — Элемент для ввода данных пользователем.

Шаг 06:

Миссия 6

 Задача: Сопоставьте теги с их свойствами.

Теги:

Ответ:

- `fieldset`
- `legend`
- `form`
- `label`

Свойства:

1. Обеспечивает семантическую группировку элементов формы.
2. Используется для описания группы элементов формы.
3. Связывает текст с конкретным полем ввода.
4. Содержит всю форму и ее элементы.

Подсказка:

Обдумайте, как разные уровни группировки влияют на структуру формы.

- **fieldset** — Обеспечивает семантическую группировку элементов формы.
- **legend** — Используется для описания группы элементов формы.
- **label** — Связывает текст с конкретным полем ввода.
- **form** — Содержит всю форму и ее элементы.

Шаг 07:

Миссия 7

 Задача: Сопоставьте ошибки с их последствиями в формах без `fieldset` и `legend`.

Ошибки:

Ответ:

- Отсутствие группировки полей
- Отсутствие заголовка группы
- Неиспользование `label` для `input`
- Неправильное вложение тегов

Последствия:

1. Пользователи не понимают, к чему относится поле
2. Сложность восприятия формы
3. Проблемы с доступностью для экранных читалок
4. Ошибка в отображении и работе формы

Подсказка:

Подумайте, как ошибки влияют на пользователей и функциональность.

- **Отсутствие группировки полей** — Сложность восприятия формы
- **Отсутствие заголовка группы** — Пользователи не понимают, к чему относится поле
- **Неиспользование label для input** — Проблемы с доступностью для экранных читалок
- **Неправильное вложение тегов** — Ошибка в отображении и работе формы

Шаг 08:

Миссия 8

Наталья создает анкету с несколькими логическими группами вопросов. Какой тег лучше использовать, чтобы семантически объединить каждую группу?

Подсказка: Этот тег помогает визуально и логически выделить блок вопросов.

Ответ:

- **fieldset**
- div
- section
- article

Шаг 09:

Миссия 9

Какой атрибут тега label связывает подпись с конкретным полем ввода?

Подсказка: Обратите внимание на то, что значение этого атрибута должно совпадать с идентификатором поля.

Ответ:

- **for**
- id
- name
- type

Шаг 10:

Миссия 10

Иван хочет, чтобы при клике на текст подписи курсор переходил в связанное поле ввода. Что нужно сделать?

Подсказка: Важно правильно связать текст с полем, чтобы улучшить удобство.

Ответ:

- **Использовать тега label с атрибутом for**
- Обернуть поле ввода в fieldset
- Добавить placeholder в input
- Использовать тег legend

Шаг 11:

Миссия 11

Что из перечисленного не является обязательным для корректной работы формы?

Подсказка: Некоторые теги нужны для семантики и удобства, но форма может работать без них.

Ответ:

- **Использование `legend` в каждой группе**
- Использование `fieldset` для группировки
- Использование `form` для обертки всех элементов
- Использование `label` для полей ввода

Шаг 12:

Миссия 12

Почему важно использовать тег `legend` внутри `fieldset` ?

Подсказка: Подумайте о доступности и структуре формы.

Ответ:

- **Для улучшения доступности и понимания группы полей**
- Чтобы форма корректно отправлялась
- Для стилизации кнопок
- Для добавления метаданных формы

Шаг 13:

Миссия 13

 Задача: Сопоставьте элементы с их допустимыми вложениями внутри `fieldset` .

Элементы:

Ответ:

- `input`
- `legend`
- `div`
- `p`

Возможные вложения:

1. Первый элемент внутри `fieldset` , описывает группу
2. Поле для ввода данных пользователя
3. Блочный контейнер для группировки
4. Текстовый блок для описания

Подсказка:

Тег `legend` должен быть всегда первым, остальные — обычные HTML элементы.

- **`legend`** — Первый элемент внутри `fieldset` , описывает группу
- **`input`** — Поле для ввода данных пользователя
- **`div`** — Блочный контейнер для группировки
- **`p`** — Текстовый блок для описания

Шаг 14:

Миссия 14

 Задача: Сопоставьте ошибки в формах без `fieldset` и `legend` с их последствиями.

Ошибки:

Ответ:

- Отсутствие группировки полей

- Отсутствие заголовка группы
- Неиспользование label для input
- Неправильное вложение тегов

Последствия:

1. Сложность восприятия формы
2. Пользователи не понимают, к чему относится поле
3. Проблемы с доступностью для экранных читалок
4. Ошибка в отображении и работе формы

🔔 Подсказка:

Подумайте, как ошибки влияют на пользователей и функциональность.

- **Отсутствие группировки полей** — Сложность восприятия формы
- **Отсутствие заголовка группы** — Пользователи не понимают, к чему относится поле
- **Неиспользование label для input** — Проблемы с доступностью для экранных читалок
- **Неправильное вложение тегов** — Ошибка в отображении и работе формы

📖 Урок 5 - Кастомизация select и checkbox

🔔 Шаг 01:

Миссия 1

Алексей хочет сделать стилизованный выпадающий список, который будет выглядеть лучше стандартного `<select>`. Какой базовый подход позволит заменить стандартный select на кастомный элемент?

Подсказка: Подумайте о том, что можно скрыть, оставив функциональность, и нарисовать свой интерфейс поверх.

Ответ:

- **Скрыть стандартный select и создать кастомный список из div и span**
- Изменить атрибуты стандартного select
- Использовать только CSS для изменения вида select
- Добавить placeholder в select

🔔 Шаг 02:

Миссия 2

Юлия создает кастомные чекбоксы и хочет, чтобы они реагировали на нажатие клавиши пробела. Что нужно добавить для корректной работы?

Подсказка: Обратите внимание на доступность и взаимодействие с клавиатурой.

Ответ:

- **Добавить tabindex и обработчик события keydown на кастомном элементе**
- Только обработчик клика мыши
- Использовать input type=checkbox без изменений
- Добавить autofocus

🔔 Шаг 03:

Миссия 3

Игорь решил скрыть стандартные чекбоксы и использовать стилизованные `<span>` для отображения состояния. Как ему синхронизировать состояние с реальными чекбоксами?

Подсказка: Нужно, чтобы визуальный элемент отражал реальное значение поля.

Ответ:

- **Обработать событие change на input и менять класс у span**
- Менять состояние span при клике на div вокруг
- Не связывать с input, а менять span отдельно
- Использовать только CSS :checked

Шаг 04:

Миссия 4

Алина хочет сделать выпадающий список, в котором выбранный элемент отображается в кастомном блоке. Какой подход позволит отследить выбор пользователя?

Подсказка: Подумайте, какое событие на select нужно использовать для обновления кастомного интерфейса.

Ответ:

- **Обработать событие change на select**
- Использовать событие click на option
- Менять состояние при focus
- Добавить событие submit формы

Шаг 05:

Миссия 5

Павел создает кастомный checkbox и хочет, чтобы он правильно работал с экранными читалками. Что нужно сделать?

Подсказка: Важна правильная связь между визуальным элементом и полем ввода.

Ответ:

- **Использовать label с атрибутом for, связав с input**
- Скрыть input полностью и не использовать label
- Добавить tabindex=-1 на кастомный элемент
- Не использовать input, только div

Шаг 06:

Миссия 6

 Задача: Сопоставьте элементы с их ролью при кастомизации чекбоксов и select.

Элементы:

Ответ:

- input type=checkbox
- span с кастомной галочкой
- div с выпадающим списком
- label

Роли:

1. Фактическое поле для ввода
2. Визуальное отображение состояния
3. Контейнер для опций select
4. Связь подписи и поля

Подсказка:

Распределите роли по их функционалу — кто отвечает за ввод, а кто — за визуализацию.

- **input type=checkbox** — Фактическое поле для ввода
- **span с кастомной галочкой** — Визуальное отображение состояния
- **div с выпадающим списком** — Контейнер для опций select
- **label** — Связь подписи и поля

🔗 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте ошибки при кастомизации select и их последствия.

Ошибки:

Ответ:

- Отсутствие обработки клавиатурных событий
- Не связанный с input кастомный элемент
- Отсутствие aria-атрибутов
- Несоответствие между состоянием input и визуальным элементом

Последствия:

1. Проблемы с доступностью для пользователей с ограничениями
2. Невозможность выбора через клавиатуру
3. Пользователь не видит реальное состояние поля
4. Снижение удобства и возможные баги в работе формы

Подсказка:

Продумайте влияние каждой ошибки на опыт пользователя.

- **Отсутствие обработки клавиатурных событий** — Невозможность выбора через клавиатуру
- **Не связанный с input кастомный элемент** — Пользователь не видит реальное состояние поля
- **Отсутствие aria-атрибутов** — Проблемы с доступностью для пользователей с ограничениями
- **Несоответствие между состоянием input и визуальным элементом** — Снижение удобства и возможные баги в работе формы

🔗 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте CSS-свойства с их ролью при стилизации кастомного select.

Свойства:

Ответ:

- appearance: none;
- position: absolute;
- z-index
- pointer-events: none;

Роли:

1. Скрыть стандартный внешний вид браузера
2. Разместить элемент поверх других

3. Управлять порядком наложения элементов
4. Запретить взаимодействие с элементом

Подсказка:

Подумайте, как управлять видимостью и взаимодействием с элементами.

- **appearance: none;** — Скрыть стандартный внешний вид браузера
- **position: absolute;** — Разместить элемент поверх других
- **z-index** — Управлять порядком наложения элементов
- **pointer-events: none;** — Запретить взаимодействие с элементом

Шаг 09:

Миссия 9

Никита работает над кастомизацией select для формы обратной связи. Он хочет заменить стандартный выпадающий список на стилизованный, но не уверен, с чего начать.

Что будет первым шагом в кастомизации select ?

Подсказка: Подумайте, как обычно скрывают стандартные элементы, чтобы начать стилизовать их по-своему.

Ответ:

- **Скрыть стандартный элемент select с помощью CSS**
- Добавить к select псевдоэлементы CSS
- Использовать только JavaScript без CSS

Шаг 10:

Миссия 10

Вика кастомизирует чекбоксы для формы регистрации. Она хочет изменить видимость стандартных галочек и сделать свои с помощью иконок.

Какой CSS-свойство поможет скрыть стандартный чекбокс, сохраняя его функциональность?

Подсказка: Элемент должен остаться в потоке, но быть невидимым.

Ответ:

- **opacity: 0;**
- **display: none;**
- **visibility: visible;**
- **position: static;**

Шаг 11:

Миссия 11

Паша разрабатывает кастомный селект с поддержкой клавиатуры. Он хочет, чтобы пользователь мог открыть меню по нажатию клавиши "Enter" или "Space".

Какая из этих реализаций поможет достичь этой цели?

Подсказка: Нужно слушать события клавиатуры и обрабатывать конкретные коды клавиш.

Ответ:

- **Добавить обработчик события `keydown` и проверять `event.key` на "Enter" и " " (пробел)**
- Использовать только событие `click`
- Игнорировать события клавиатуры, работать только мышью

🔗 Шаг 12:

Миссия 12

Данил хочет улучшить кастомные чекбоксы, но путается в терминологии. Помогите ему сопоставить понятия и описания:

Ответ:

- label
- input[type=checkbox]
- checked
- for атрибут в <label>

Подсказка: Связь между текстом и элементом важна для кликабельности.

- **label** — Элемент, который содержит текст и связывается с чекбоксом.
- **input[type=checkbox]** — Форменный элемент для создания флажка, который можно отметить или снять.
- **checked** — Атрибут, указывающий, что чекбокс выбран.
- **for атрибут в** — Связывает label с input по id, чтобы клик по тексту активировал чекбокс.

🔗 Шаг 13:

Миссия 13

Костя работает с кастомным селектом и хочет понять, как связаны методы и свойства для управления состоянием.

Сопоставьте методы/свойства с их назначением:

Ответ:

- open()
- close()
- toggle()
- isOpen

Подсказка: Одно свойство отвечает за состояние, остальные — за действия.

- **open()** — Открывает выпадающее меню.
- **close()** — Закрывает выпадающее меню.
- **toggle()** — Переключает состояние меню между открытым и закрытым.
- **isOpen** — Булево свойство, показывающее, открыто ли меню.

🔗 Шаг 14:

Миссия 14

Света изучает приемы кастомизации чекбоксов с использованием псевдоэлементов CSS.

Соотнесите псевдоэлементы с их описанием:

Ответ:

- ::before
- ::after
- :checked
- :hover

Подсказка: Псевдоэлементы добавляют контент, псевдоклассы — стили в зависимости от состояния.

- **::before** — Добавляет содержимое перед содержимым элемента.
- **:after** — Добавляет содержимое после содержимого элемента.
- **:checked** — Псевдокласс для отмеченного чекбокса.
- **:hover** — Псевдокласс при наведении курсора мыши.

🔗 Шаг 15:

Миссия 15

Алексей создает кастомный select и хочет реализовать правильный порядок действий при открытии и закрытии меню.

Расставьте шаги в правильной последовательности:

1. Проверка текущего состояния меню
2. Изменение видимости меню
3. Обработка событий клика и клавиатуры
4. Обновление состояния переменной, отражающей открытость меню

Подсказка: Подумайте, что нужно сделать перед изменением состояния, а что после.

Ответ:

- Обработка событий клика и клавиатуры
- Проверка текущего состояния меню
- Изменение видимости меню
- Обновление состояния переменной, отражающей открытость меню

🔗 Шаг 16:

Миссия 16

Дима работает над кастомизацией чекбоксов и хочет определить правильный порядок стилей для псевдоэлементов.

Расположите по порядку, в каком порядке должны применяться стили:

1. Стили для :hover
2. Стили для :checked
3. Стили для ::before
4. Стили для базового input[type=checkbox]

Подсказка: Учитывайте специфику псевдоэлементов и псевдоклассов.

Ответ:

- Стили для базового input[type=checkbox]
- Стили для ::before
- Стили для :checked
- Стили для :hover

🔗 Шаг 17:

Миссия 17

Катя хочет добавить кастомному select эффект плавного раскрытия меню с помощью CSS.

Какое свойство CSS лучше использовать для анимации раскрытия списка?

Подсказка: Подумайте о свойствах, которые можно анимировать для плавного появления.

Ответ:

- **transition** с изменением **opacity** и **max-height**
- **animation** с изменением цвета текста
- **transform** с изменением **translateX**

 **Шаг 18:**

Миссия 18

Сергей хочет сделать кастомные чекбоксы доступными для пользователей с ограничениями по зрению.

Какой элемент HTML поможет обеспечить доступность?

Подсказка: Используйте связку для управления фокусом и озвучивания.

Ответ:

- **Использовать элемент `label` атрибутом `for`**
- Только `div` без связки с `input`
- Использовать скрытый текст без `label`

 **Шаг 19:**

Миссия 19

Ирина разбирается в событиях JavaScript для управления кастомными `select` и `checkbox`.

Сопоставьте события с их назначением:

Ответ:

- `change`
- `click`
- `focus`
- `blur`

Подсказка: Некоторые события срабатывают при выборе, другие — при потере или получении фокуса.

- **`change`** — Срабатывает, когда значение элемента изменяется.
- **`click`** — Срабатывает при клике по элементу.
- **`focus`** — Срабатывает, когда элемент получает фокус.
- **`blur`** — Срабатывает, когда элемент теряет фокус.

Модуль 3 - 🏰 Таблицы и данные

Урок 1 - Разметка `thead`, `tbody`, `tfoot`

 **Шаг 01:**

Миссия 1

Аня, фронтенд-разработчица, создает таблицу для отчета и хочет правильно структурировать заголовки колонок. Какой тег HTML используется для обозначения заголовка таблицы?

Подсказка: Заголовки таблиц обычно помещают в отдельный блок, который предшествует основным данным.

Ответ:

- **`thead`**
- `tbody`

- tfoot
- caption

Шаг 02:

Миссия 2

Максим делает таблицу продаж и хочет выделить подвал таблицы с итоговыми значениями. Какой тег ему нужно использовать для этого?

Подсказка: Итоговые данные обычно располагаются в конце таблицы.

Ответ:

- **tfoot**
- thead
- tbody
- tfootbody

Шаг 03:

Миссия 3

Екатерина пишет код и хочет разбить содержимое таблицы на основную часть с данными и вспомогательную часть с заголовками и подвалом. Какой тег должен содержать основной контент таблицы?

Подсказка: Это тело таблицы, где располагаются все основные данные.

Ответ:

- **tbody**
- thead
- tfoot
- body

Шаг 04:

Миссия 4

Даниил замечает, что некоторые браузеры неправильно отображают таблицу без разделения на thead, tbody и tfoot. Почему важно использовать эти теги?

Подсказка: Они помогают браузеру понять структуру таблицы и корректно ее отобразить.

Ответ:

- **Для логической структуры и правильного отображения данных**
- Для стилизации таблицы с помощью CSS
- Для ускорения загрузки страницы
- Для создания анимаций в таблице

Шаг 05:

Миссия 5

Лена хочет, чтобы заголовки таблицы повторялись на каждой странице при печати. Какой тег поможет браузеру распознать заголовочную часть для этого?

Подсказка: Заголовки, которые должны повторяться, обычно оформляют специальным тегом.

Ответ:

- **thead**
- tfoot

- tbody
- caption

🔗 Шаг 06:

Миссия 6

📖 Вадим организует структуру таблицы и хочет понять, что соответствует каждому тегу. Сопоставьте теги с их описаниями.

Теги:

Ответ:

- thead
- tbody
- tfoot

Описание:

1. Основная часть с данными таблицы
2. Подвал таблицы с итогами или заметками
3. Заголовочная часть с названиями колонок

Подсказка: Представьте таблицу как книгу: титульный лист, содержание и заключение.

- **thead** — Заголовочная часть с названиями колонок
- **tbody** — Основная часть с данными таблицы
- **tfoot** — Подвал таблицы с итогами или заметками

🔗 Шаг 07:

Миссия 7

📖 Света работает над отчетом с таблицей и должна правильно сопоставить функции тегов.

Теги:

Ответ:

- thead
- tbody
- tfoot

Функции:

1. Содержит основные данные с записями
2. Используется для группировки итоговых значений
3. Определяет строки с заголовками колонок

Подсказка: Задумайтесь, где располагается каждая часть в таблице и что она показывает.

- **thead** — Определяет строки с заголовками колонок
- **tbody** — Содержит основные данные с записями
- **tfoot** — Используется для группировки итоговых значений

🔗 Шаг 08:

Миссия 8

Иван — начинающий фронтенд-разработчик, который делает таблицу с данными о сотрудниках. Он хочет структурировать таблицу так, чтобы заголовки всегда были отдельной частью. Какой тег ему следует использовать для группировки заголовков таблицы?

Подсказка: Заголовки таблицы обычно располагаются в верхней части и отделяются от данных.

Ответ:

- **thead**
- tbody
- tfoot
- tr

 **Шаг 09:**

Миссия 9

Анна верстает таблицу с итогами продаж. Ей нужно, чтобы итоговая строка всегда оставалась внизу таблицы и содержала общие суммы. Какой тег лучше использовать для этой цели?

Подсказка: Итоговые данные часто помещают в отдельный блок, который находится после основного тела таблицы.

Ответ:

- **tfoot**
- thead
- tbody
- th

 **Шаг 10:**

Миссия 10

Пётр создает таблицу с расписанием. Он хочет, чтобы данные о каждом дне недели были в отдельном блоке, чтобы легко управлять стилями. Какой тег следует использовать для содержимого таблицы?

Подсказка: Основные данные таблицы обычно содержатся в центральной части.

Ответ:

- **tbody**
- thead
- tfoot
- tr

 **Шаг 11:**

Миссия 11

Мария хочет улучшить структуру своей таблицы и правильно использовать теги. Какие теги относятся к группировке строк таблицы?

Подсказка: Эти теги помогают разделять части таблицы на логические блоки.

Ответ:

- **thead**
- **tbody**
- **tfoot**
- td

 **Шаг 12:**

Миссия 12

Вася заметил, что некоторые таблицы в браузерах отображаются неправильно, потому что не указаны блоки `thead`, `tbody` и `tfoot`. Что происходит, если не использовать эти теги?

Подсказка: Браузеры пытаются сами определить структуру, но это может привести к неожиданным результатам.

Ответ:

- **Браузер может неправильно разделить заголовки, тело и футер таблицы**
- Таблица перестанет отображаться вообще
- Теги `thead`, `tbody` и `tfoot` автоматически создаются браузером
- Строки таблицы будут подсвечены красным цветом

Шаг 13:

Миссия 13

 Помогите Артему сопоставить теги таблицы с их назначением.

Теги:

Ответ:

- `thead`
- `tbody`
- `tfoot`

Назначения:

1. Основной контент таблицы
2. Заголовок таблицы
3. Нижняя часть с итогами или примечаниями

Подсказка: Заголовки всегда сверху, тело — посередине, итоги — внизу.

- **`thead`** — Заголовок таблицы
- **`tbody`** — Основной контент таблицы
- **`tfoot`** — Нижняя часть с итогами или примечаниями

Шаг 14:

Миссия 14

 Сопоставьте теги таблицы с их правильным описанием.

Ответ:

- `thead`
 - `tbody`
 - `tfoot`
1. Содержит основные данные таблицы
 2. Используется для группировки заголовков таблицы
 3. Часто содержит итоговые или вспомогательные данные

Подсказка: Помните расположение каждого блока в таблице и его смысл.

- **`thead`** — Используется для группировки заголовков таблицы
- **`tbody`** — Содержит основные данные таблицы
- **`tfoot`** — Часто содержит итоговые или вспомогательные данные

🔗 Шаг 15:

Миссия 15

🔗 Сопоставьте теги с их особенностями в HTML-таблице:

Ответ:

- `thead`
 - `tbody`
 - `tfoot`
1. Может повторяться на каждой странице при печати
 2. Основной контейнер для данных
 3. Используется для повторяющихся заголовков

Подсказка: Подумайте, какой блок обычно выводится сверху и может дублироваться при разбиении таблицы.

- **`thead`** — Используется для повторяющихся заголовков
- **`tbody`** — Основной контейнер для данных
- **`tfoot`** — Может повторяться на каждой странице при печати

🔗 Шаг 16:

Миссия 16

🔗 Соотнесите теги с их ролью в структурировании таблиц:

Ответ:

- `thead`
 - `tbody`
 - `tfoot`
1. Обязателен для оформления заголовков
 2. Может содержать несколько строк с данными
 3. Часто содержит строку с итогами или сносками

Подсказка: Вспомните, что `thead` и `tfoot` располагаются на границах таблицы.

- **`thead`** — Обязателен для оформления заголовков
- **`tbody`** — Может содержать несколько строк с данными
- **`tfoot`** — Часто содержит строку с итогами или сносками

🔗 Шаг 17:

Миссия 17

🔗 Сопоставьте теги с их свойствами в работе с таблицами:

Ответ:

- `thead`
 - `tbody`
 - `tfoot`
1. Отвечает за визуальное разделение заголовка
 2. Основной блок с контентом таблицы
 3. Блок с информацией, которая отображается после данных

Подсказка: Помните, что правильное разделение облегчает поддержку кода.

- **thead** — Отвечает за визуальное разделение заголовка
- **tbody** — Основной блок с контентом таблицы
- **tfoot** — Блок с информацией, которая отображается после данных

Шаг 18:

Миссия 18

Расположите последовательность действий для правильного построения таблицы с `thead`, `tbody` и `tfoot`:

1. Добавление строк с итогами
2. Группировка строк с заголовками
3. Заполнение основного содержимого таблицы
4. Создание таблицы и определение структуры

Подсказка: Начинайте с общей структуры, затем добавляйте содержимое и итоги.

Ответ:

- Создание таблицы и определение структуры
- Группировка строк с заголовками
- Заполнение основного содержимого таблицы
- Добавление строк с итогами

Шаг 19:

Миссия 19

Упорядочите шаги по корректной вставке блоков `thead`, `tbody` и `tfoot` в таблицу:

1. Объявить тег `thead` и вставить заголовочные строки
2. Создать основной блок `tbody` с данными
3. Добавить footer таблицы с помощью `tfoot`
4. Объявить тег `table` и задать атрибуты

Подсказка: Не забывайте, что таблица начинается с тега `table`, а затем идут её части.

Ответ:

- Объявить тег `table` и задать атрибуты
- Объявить тег `thead` и вставить заголовочные строки
- Создать основной блок `tbody` с данными
- Добавить footer таблицы с помощью `tfoot`

Урок 2 - Атрибуты `scope` и `headers`

Шаг 01:

Миссия 1

 Артём строит сложную таблицу и путается в атрибутах. Сопоставьте атрибут с его назначением.

Подсказка: Вспомните: `scope` для заголовков, `headers` для связи с `id`.

Ответ:

- **scope** — Определяет, относится ли заголовок к строке, колонке или группе
- **headers** — Указывает `id` заголовков, с которыми связана ячейка
- **rowspan** — Объединяет ячейки по вертикали

- **colspan** — Объединяет ячейки по горизонтали

🔗 Шаг 02:

Миссия 2

💻 Елена пишет код для таблицы с зарплатами. Сопоставьте ошибки с их последствиями.

Подсказка: Ошибки в атрибутах влияют на доступность, а не на внешний вид.

Ответ:

- **Пропущен атрибут score** — Скринридер не поймёт связь заголовков и данных
- **Использован headers без id в th** — Ячейка не сможет найти нужный заголовок
- **Неправильное значение score** — Заголовок применяется не к тем ячейкам
- **Нет связи между td и th** — Проблемы с доступностью для слабовидящих

🔗 Шаг 03:

Миссия 3

Андрей готовит таблицу для отчета. Он заметил атрибут score . Какое основное назначение у него?

Подсказка: Думай о том, как экранные читалки понимают заголовки в таблице.

Ответ:

- **Помогает связать заголовок таблицы с соответствующими ячейками**
- Изменяет визуальный стиль заголовка
- Устанавливает ширину колонки
- Добавляет всплывающую подсказку

🔗 Шаг 04:

Миссия 4

Мария верстает таблицу расписания. Какое значение score нужно поставить для заголовка столбца «Время»?

Подсказка: Подумай: строка или колонка?

Ответ:

- **col**
- row
- auto

🔗 Шаг 05:

Миссия 5

Иван создает таблицу заказов. В первой строке у него названия категорий. Какое значение атрибута score правильное для этих заголовков?

Подсказка: Заголовки сверху — это...

Ответ:

- **col**
- row
- group

Шаг 06:

Миссия 6

Оля делает таблицу с именами сотрудников. В первом столбце — фамилии. Какое значение атрибута `scope` выбрать для этих заголовков?

Подсказка: Вспомни: первый столбец описывает строки.

Ответ:

- **row**
- col
- all

Шаг 07:

Миссия 7

Светлана тестирует таблицу на доступность. Какой атрибут лучше использовать, если одна ячейка заголовка относится к ячейкам, расположенным не подряд?

Подсказка: Когда простого `scope` уже мало.

Ответ:

- **headers**
- scope
- rowspan
- colspan

Шаг 08:

Миссия 8

Артем связывает сложную таблицу с подзаголовками. Что должен содержать атрибут `headers` у ячейки?

Подсказка: Это список чего-то уникального...

Ответ:

- **ID заголовочных ячеек, с которыми связана данная ячейка**
- Названия классов CSS
- Значения атрибута `style`
- Название таблицы

Шаг 09:

Миссия 9

 Сопоставьте значения атрибута `scope` с их назначением.

Подсказка: Подумай о направлениях: строки, колонки, группы.

Ответ:

- **row** — Заголовок относится к строке
- **col** — Заголовок относится к колонке
- **rowgroup** — Заголовок описывает группу строк
- **colgroup** — Заголовок описывает группу колонок

Шаг 10:

Миссия 10

Анна готовит таблицу статистики. Сопоставьте атрибуты с их описанием.

Подсказка: Один отвечает за простую структуру, другой — за сложные связи.

Ответ:

- **scope** — Простая привязка заголовка к строке или колонке
- **headers** — Указание ID нескольких заголовков для сложных таблиц

🔗 Шаг 11:

Миссия 11

Петр хочет понять, зачем нужны атрибуты. Сопоставьте их с целями.

Подсказка: Смотри на ключевое слово «доступность».

Ответ:

- **scope** — Упрощает навигацию по таблице для скринридеров
- **headers** — Позволяет точно связать данные с нужными заголовками

🔗 Шаг 12:

Миссия 12

Лена изучает ошибки в вёрстке. Сопоставьте проблему с правильным описанием.

Подсказка: Ошибки часто связаны с неправильным использованием.

Ответ:

- **scope в обычной ячейке td** — Некорректное использование — атрибут должен быть в th
- **headers без ID у заголовка** — Нет связи, скринридер не поймет

🔗 Шаг 13:

Миссия 13

Расположите шаги добавления атрибута scope к таблице.

Подсказка: Сначала находишь заголовки, потом определяешь направление.

Ответ:

- Определить, какие ячейки являются заголовками
- Понять, относится ли заголовок к строке, колонке или группе
- Добавить атрибут scope с нужным значением
- Проверить работу через скринридер

🔗 Шаг 14:

Миссия 14

Саша добавляет headers для сложной таблицы. Расположите действия в правильном порядке.

Подсказка: Начинать с разметки заголовков.

Ответ:

- Присвоить ID заголовочным ячейкам
- Определить, какие ячейки данных к ним относятся
- Указать атрибут headers в нужных ячейках
- Протестировать через скринридер

Шаг 15:

Миссия 15

Дима оформляет отчёт. Расположите шаги для работы с `score` и `headers` в одной таблице.

Подсказка: Используйте простое там, где можно, и `headers` там, где нужно.

Ответ:

- Добавить `score` для простых заголовков
- Назначить ID для сложных заголовков
- Использовать `headers` для связи с несколькими заголовками
- Проверить итоговую доступность

Шаг 16:

Миссия 16

Финальное испытание: Расположите шаги создания доступной таблицы с данными.

Подсказка: Думай как архитектор: структура, атрибуты, проверка.

Ответ:

- Определить структуру таблицы
- Проставить `score` в простых случаях
- Добавить ID заголовкам
- Прописать `headers` там, где нужно
- Протестировать доступность

Урок 3 - Сложные таблицы с объединением

Шаг 01:

Миссия 1

 Аня верстает таблицу меню. Она хочет объединить две ячейки по горизонтали, чтобы написать «Горячие напитки». Какой атрибут нужно использовать?

Подсказка: Этот атрибут управляет объединением колонок.

Ответ:

- **`colspan`**
- `rowspan`
- `score`

Шаг 02:

Миссия 2

 Игорь делает таблицу расписания автобусов. Нужно объединить ячейки по вертикали для остановки «Центр» (она одна на два рейса). Какой атрибут он выберет?

Подсказка: Думайте про вертикаль.

Ответ:

- **`rowspan`**
- `colspan`
- `headers`

Шаг 03:

Миссия 3

 Кира делает таблицу с результатами тестов. Она объединила три колонки в одну, но данные стали некорректно отображаться. Что могло пойти не так?

Подсказка: Часто ошибка — неправильное значение атрибута.

Ответ:

- **Неправильно указано значение colspan**
- Использован rowspan вместо colspan
- Добавлен scope вместо colspan
- Пропущен headers

Шаг 04:

Миссия 4

 Артём учит стажёров объединять ячейки. Сопоставьте атрибут с его назначением.

Подсказка: Один отвечает за строки, другой за колонки.

Ответ:

- **colspan** — Объединяет ячейки по горизонтали
- **rowspan** — Объединяет ячейки по вертикали
- **scope** — Определяет связь заголовка с ячейками
- **headers** — Связывает td с th через id

Шаг 05:

Миссия 5

 Елена изучает ошибки новичков при объединении ячеек. Сопоставьте ошибку с её результатом.

Подсказка: Ошибки могут влиять на отображение таблицы.

Ответ:

- **Превышено значение colspan** — Таблица «разъезжается» по ширине
- **Превышено значение rowspan** — Сбивается структура строк
- **Нет colspan при объединённом заголовке** — Заголовок занимает только одну колонку
- **Использован rowspan вместо colspan** — Данные отображаются неправильно

Шаг 06:

Миссия 6

 Лена объясняет новичкам, как пошагово объединять ячейки. Расставьте действия в правильном порядке.

Подсказка: Сначала определяем структуру, потом объединяем.

Ответ:

- Определить, какие ячейки нужно объединить
- Выбрать правильный атрибут (colspan или rowspan)
- Указать число объединяемых ячеек
- Проверить визуально таблицу

Шаг 07:

Миссия 7

 Дима готовит таблицу отчёта. Нужно расставить этапы объединения строк.

Подсказка: Думайте про вертикальное объединение.

Ответ:

- Определить ячейки для объединения
- Добавить атрибут `rowspan`
- Задать нужное число строк
- Проверить отображение в браузере

Шаг 08:

Миссия 8

Андрей верстает таблицу расписания и хочет объединить две ячейки по горизонтали. Какой атрибут для этого нужен?

Подсказка: Атрибут управляет количеством столбцов, которые занимает ячейка.

Ответ:

- **`colspan`**
- `rowspan`
- `merge`
- `combine`

Шаг 09:

Миссия 9

Мария делает отчёт и хочет объединить ячейки в колонке по вертикали. Какой атрибут ей поможет?

Подсказка: Он задаёт количество строк, которые будет занимать ячейка.

Ответ:

- **`rowspan`**
- `colspan`
- `align`

Шаг 10:

Миссия 10

Иван оформляет таблицу. В какой тег помещаются объединяемые ячейки?

Подсказка: Эти ячейки находятся внутри строки.

Ответ:

- **или**
-
-

Шаг 11:

Миссия 11

Оля строит таблицу бюджета. Она хочет объединить первую строку, чтобы там была общая надпись «Отчёт». Как правильно?

Подсказка: Используй горизонтальное объединение.

Ответ:

- **Отчёт**
- Отчёт
- Отчёт
- Отчёт

Шаг 12:

Миссия 12

Никита верстает календарь. У него есть одна ячейка «Понедельник», которая должна занимать сразу две строки. Как записать?

Подсказка: Используй вертикальное объединение.

Ответ:

- **Понедельник**
- Понедельник
- Понедельник

Шаг 13:

Миссия 13

Светлана хочет объединить ячейку заголовка таблицы так, чтобы она растянулась на 4 колонки. Какой код подходит?

Подсказка: Заголовки таблицы пишут в специальном теге.

Ответ:

- **Заголовок**
- Заголовок
- Заголовок
- Заголовок

Шаг 14:

Миссия 14

Артем делает таблицу расписания поездов. Какая комбинация атрибутов позволит объединить ячейку сразу по строкам и по колонкам?

Подсказка: Эти атрибуты можно использовать вместе.

Ответ:

- **colspan и rowspan одновременно**
- merge и span
- width и height
- align и valign

Шаг 15:

Миссия 15

Катя изучает большие таблицы. В какой части HTML-кода обычно используют объединение ячеек чаще всего?

Подсказка: Это может быть верхняя часть таблицы.

Ответ:

- B
- B
- B

Шаг 16:

Миссия 16

 Сопоставьте атрибут объединения ячеек с его назначением.

Подсказка: Один работает по горизонтали, другой — по вертикали.

Ответ:

- **colspan** — Объединяет ячейки по горизонтали
- **rowspan** — Объединяет ячейки по вертикали

Шаг 17:

Миссия 17

Анна проверяет знания. Сопоставьте тег с его функцией.

Подсказка: Один отвечает за строки, другой — за ячейки.

Ответ:

- — Создаёт строку в таблице
- — Создаёт обычную ячейку
- — Создаёт заголовочную ячейку

Шаг 18:

Миссия 18

Петр изучает структуру. Сопоставьте секции таблицы с их ролью.

Подсказка: Вспомни, где обычно находятся заголовки, данные и итоги.

Ответ:

- — Содержит заголовки столбцов
- — Содержит основные данные
- — Содержит итоговую информацию

Шаг 19:

Миссия 19

Миша анализирует ошибки. Сопоставьте ошибку с её последствием.

Подсказка: Подумай, что произойдёт, если объединение указано неверно.

Ответ:

- **Неверный colspan** — Ячейка вылезает за пределы строки
- **Неверный rowspan** — Ячейка ломает структуру столбца

Урок 4 - Подписи и стилизация таблиц

Шаг 01:

Миссия 1

Оля хочет выровнять подпись таблицы по низу. Какой CSS-свойство нужно использовать?

Подсказка: Для этого есть специальное свойство, связанное с расположением подписи.

Ответ:

- **caption-side: bottom;**
- vertical-align: bottom;
- text-align: bottom;

Шаг 02:

Миссия 2

Никита решил сделать подпись таблицы курсивной. Какой CSS нужно применить?

Подсказка: Вспомни свойство для начертания текста.

Ответ:

- **caption { font-style: italic; }**
- caption { style: italic; }
- caption { text-decoration: italic; }

Шаг 03:

Миссия 3

Светлана хочет придать подписи таблицы особый цвет. Какой способ правильный?

Подсказка: Делается через свойство, которое управляет цветом текста.

Ответ:

- **caption { color: blue; }**
- caption { text-color: blue; }
- caption { font-color: blue; }

Шаг 04:

Миссия 4

Артем оформляет таблицу и думает: можно ли внутри использовать теги HTML для оформления текста?

Подсказка: Вспомни, является ли блочным контейнером.

Ответ:

- **Да, можно использовать встроенные теги форматирования**
- Нет, может содержать только текст без тегов

Шаг 05:

Миссия 5

Катя изучает доступность. Как помогает пользователям со скринридерами?

Подсказка: Подумай: что даёт подпись таблице с точки зрения смысла?

Ответ:

- **Дает краткое описание содержимого таблицы**
- Добавляет стили таблице
- Увеличивает шрифт первой строки

Шаг 06:

Миссия 6

 Сопоставьте CSS-свойство с его эффектом для .

Подсказка: Свойства отвечают за цвет, выравнивание и шрифт.

Ответ:

- **color** — Изменяет цвет текста подписи
- **caption-side** — Определяет расположение подписи относительно таблицы
- **font-style** — Меняет стиль начертания текста
- **text-align** — Выравнивает текст внутри подписи

Шаг 07:

Миссия 7

Анна тестирует таблицы. Сопоставьте тег или атрибут с его функцией.

Подсказка: Один задаёт подпись, другой — стиль.

Ответ:

- — Добавляет подпись к таблице
- **caption-side** — Управляет расположением подписи

Шаг 08:

Миссия 8

Петр хочет понять различия. Сопоставьте понятие с описанием.

Подсказка: Одна часть — HTML, другая — CSS.

Ответ:

- **HTML** — Задаёт смысловую подпись таблице
- **CSS оформление** — Отвечает за внешний вид подписи

Шаг 09:

Миссия 9

Миша анализирует код. Сопоставьте ошибку с её последствием.

Подсказка: Ошибки часто связаны с отсутствием стиля или нарушением структуры.

Ответ:

- **Отсутствие** — Таблица теряет смысловое описание для скринридеров
- **Неверное использование caption-side** — Подпись отображается не там, где ожидалось

Шаг 10:

Миссия 10

Лена изучает особенности стилизации. Сопоставьте свойство с его примером.

Подсказка: Каждое свойство связано с конкретным значением.

Ответ:

- **caption-side** — caption-side: top;
- **font-weight** — font-weight: bold;
- **font-style** — font-style: italic;
- **color** — color: red;

🔗 Шаг 11:

Миссия 11

Расположите шаги для добавления подписи к таблице.

Подсказка: Сначала структура, потом оформление.

Ответ:

- Открыть тег
- Добавить тег с текстом
- Создать содержимое таблицы
- Применить стили для подписи

🔗 Шаг 12:

Миссия 12

Саша оформляет таблицу. Расположите шаги для выравнивания подписи снизу.

Подсказка: Вспомни про CSS-свойство.

Ответ:

- Добавить тег
- Задать текст подписи
- Написать CSS правило `caption-side: bottom;`
- Проверить отображение

🔗 Шаг 13:

Миссия 13

Дима оформляет отчёт. Расположите шаги стилизации подписи таблицы.

Подсказка: Сначала напиши HTML, потом CSS.

Ответ:

- Создать таблицу с тегом
- Добавить текст подписи
- Задать цвет и стиль текста через CSS
- Проверить результат в браузере

🔗 Шаг 14:

Миссия 14

Финальное испытание: Расположите шаги для создания доступной и стильной таблицы с подписью.

Подсказка: Думай как архитектор: структура, подпись, стилизация, проверка.

Ответ:

- Создать таблицу с данными
- Добавить тег с описанием

- Применить стили для подписи
- Установить расположение подписи через caption-side
- Протестировать итоговое отображение

Модуль 4 - Интернационализация

Урок 1 - Атрибут lang и языковые блоки

Шаг 01:

Миссия 1

Анна верстает страницу и хочет указать основной язык документа. Какой атрибут использовать?

Подсказка: Атрибут пишется в теге .

Ответ:

- **lang**
- language
- xml:lang

Шаг 02:

Миссия 2

Елена хочет указать язык для подписи изображения на итальянском. Как оформить тег?

Подсказка: Атрибут lang применяется к тегу

.

Ответ:

- **Descrizione**
- Descrizione
- Descrizione

Шаг 03:

Миссия 3

Светлана верстает форму на английском и французском. Как пометить кнопки на двух языках?

Подсказка: Каждый текст кнопки — отдельный языковой блок.

Ответ:

- **SubmitSoumettre**
- SubmitSoumettre
- SubmitSoumettre

Шаг 04:

Миссия 4

 Сопоставьте язык с его правильным кодом для атрибута lang.

Подсказка: Используйте стандартизированные коды ISO 639-1.

Ответ:

- **Русский** — ru
- **Английский** — en
- **Французский** — fr

- **Японский** — ja

Шаг 05:

Миссия 5

Анна создает страницу и хочет указать основной язык документа. Какой атрибут использовать?

Подсказка: Атрибут пишется в теге .

Ответ:

- lang
- language
- xml:lang

Шаг 06:

Миссия 6

 Сопоставьте язык с его правильным кодом для атрибута lang.

Подсказка: Используйте стандартизированные коды ISO 639-1.

Ответ:

- **Русский** — ru
- **Английский** — en
- **Французский** — fr
- **Японский** — ja

Шаг 07:

Миссия 7

Расположите действия по интернационализации текста на странице в правильном порядке:

1. Добавить основной язык документа в тег
2. Пометить отдельные абзацы с другим языком с помощью lang
3. Добавить подписи к изображениям с указанием языка
4. Проверить корректность кода и отображение языковых блоков

Подсказка: Сначала указываем общий язык, затем локальные блоки, и только после этого проверяем.

Ответ:

- Добавить основной язык документа в тег
- Пометить отдельные абзацы с другим языком с помощью lang
- Добавить подписи к изображениям с указанием языка
- Проверить корректность кода и отображение языковых блоков

Урок 2 - RTL и LTR-направления текста

Шаг 01:

Миссия 1

Алия создает веб-страницу на арабском языке. Какой атрибут нужно использовать для направления текста справа налево?

Подсказка: Атрибут указывает направление текста.

Ответ:

- `dir='rtl'`
- `lang='ar'`
- `direction='rtl'`

Шаг 02:

Миссия 2

Даниил пишет статью на английском и хочет, чтобы текст отображался слева направо. Что он должен указать?

Подсказка: LTR — стандартное направление для большинства языков.

Ответ:

- `dir='ltr'`
- `dir='rtl'`
- `lang='en'`

Шаг 03:

Миссия 3

 Сопоставьте HTML-атрибут с его назначением.

Подсказка: Атрибуты могут управлять направлением или языком текста.

Ответ:

- **dir** — Определяет направление текста (ltr или rtl)
- **lang** — Указывает язык текста
- **xml:lang** — Используется в XML-документах для указания языка
- **class** — Не влияет на направление текста

Шаг 04:

Миссия 4

 Сопоставьте элемент HTML с примером правильного использования RTL или LTR.

Подсказка: Контейнеры текста могут наследовать направление.

Ответ:

- **Абзац на арабском** — مرحبا
- **Заголовок на английском** — Welcome
- **Блок на иврите** — שלום עולם
- **Параграф на русском** — Привет мир

Шаг 05:

Миссия 5

Расположите действия для корректного отображения RTL и LTR на странице в правильном порядке:

1. Установить основной язык и направление документа
2. Пометить отдельные блоки текста с отличающимся направлением
3. Проверить визуальное отображение текста
4. При необходимости скорректировать CSS для поддержки RTL

Подсказка: Сначала глобальные настройки, потом локальные блоки, затем проверка и стили.

Ответ:

- Установить основной язык и направление документа
- Пометить отдельные блоки текста с отличающимся направлением
- Проверить визуальное отображение текста
- При необходимости скорректировать CSS для поддержки RTL

 Шаг 06:

Миссия 6

Алия создает веб-страницу на арабском языке. Какой атрибут HTML нужно использовать для направления текста справа налево?

Подсказка: Атрибут определяет направление текста.

Ответ:

- **dir='rtl'**
- lang='ar'
- direction='rtl'

 Шаг 07:

Миссия 7

Даниил пишет статью на английском языке и хочет, чтобы текст отображался слева направо. Какой атрибут следует использовать?

Подсказка: LTR — стандартное направление для большинства языков.

Ответ:

- **dir='ltr'**
- dir='rtl'
- lang='en'

 Шаг 08:

Миссия 8

 Сопоставьте HTML-атрибут с его назначением.

Подсказка: Атрибуты могут управлять направлением или языком текста.

Ответ:

- **dir** — Определяет направление текста (ltr или rtl)
- **lang** — Указывает язык текста
- **xml:lang** — Используется в XML-документах для указания языка
- **class** — Не влияет на направление текста

 Шаг 09:

Миссия 9

 Сопоставьте элемент HTML с примером правильного использования RTL или LTR.

Подсказка: Контейнеры текста могут наследовать направление.

Ответ:

- **Абзац на арабском** — مرحبا
- **Заголовок на английском** — Welcome
- **Блок на иврите** — שלום עולם

- **Параграф на русском** — Привет мир

Шаг 10:

Миссия 10

Расположите действия для корректного отображения RTL и LTR на странице в правильном порядке:

1. Установить основной язык и направление документа
2. Пометить отдельные блоки текста с отличающимся направлением
3. Проверить визуальное отображение текста
4. При необходимости скорректировать CSS для поддержки RTL

Подсказка: Сначала глобальные настройки, потом локальные блоки, затем проверка и стили.

Ответ:

- Установить основной язык и направление документа
- Пометить отдельные блоки текста с отличающимся направлением
- Проверить визуальное отображение текста
- При необходимости скорректировать CSS для поддержки RTL

Шаг 11:

Миссия 11

Программист Иван хочет сделать часть текста на арабском языке внутри английского документа. Какой атрибут лучше всего использовать?

Подсказка: Подумайте, как указать направление для отдельного блока.

Ответ:

- `dir='rtl'`
- `lang='en'`
- `dir='ltr'`

Урок 3 - Метаданные для локализации

Шаг 01:

Миссия 1

Алекс создаёт страницу и хочет указать основной язык документа. Какой HTML-атрибут для этого используют?

Подсказка: Этот атрибут помогает поисковикам и браузерам понимать язык текста.

Ответ:

- `lang='en'`
- `dir='ltr'`
- `charset='UTF-8'`

Шаг 02:

Миссия 2

Виктория хочет, чтобы поисковые системы знали, что её сайт на французском языке. Какой атрибут следует использовать?

Подсказка: Подумайте о том, как указать язык для всего документа.

Ответ:

- `lang='fr'`
- `dir='rtl'`
- `meta name='language'`

Шаг 03:

Миссия 3

Иван хочет, чтобы браузеры корректно отображали кириллицу. Какой мета-тег отвечает за кодировку?

Подсказка: Кодировка указывается в `` и обычно стоит в ``.

Ответ:

-
-
-

Шаг 04:

Миссия 4

Мария хочет, чтобы её сайт был правильно отображен на мобильных устройствах. Какой метатег ей нужен?

Подсказка: Этот метатег часто содержит width и initial-scale.

Ответ:

-
-
- `lang='en'`

Шаг 05:

Миссия 5

 Сопоставьте метаданные с их назначением.

Подсказка: Некоторые метатеги помогают браузеру, другие — поисковикам.

Ответ:

- **lang** — Указывает основной язык документа
- **charset** — Задаёт кодировку текста
- **viewport** — Оптимизирует отображение на мобильных устройствах
- **description** — Кратко описывает содержание страницы для поисковиков

Шаг 06:

Миссия 6

 Сопоставьте метатеги с правильными примерами использования.

Подсказка: Обратите внимание на синтаксис и назначение каждого тега.

Ответ:

- **Указать язык** —
- **Кодировка UTF-8** —
- **Описание страницы** —
- **Адаптивность** —

Шаг 07:

Миссия 7

 Сопоставьте задачу с подходящим метатегом.

Подсказка: Метатеги помогают как пользователю, так и поисковым системам.

Ответ:

- **Обеспечить корректное отображение символов —**
- **Указать язык документа —**
- **Оптимизация под мобильные —**
- **Описание для поисковиков —**

Шаг 08:

Миссия 8

Дмитрий хочет добавить метаданные, которые не видны пользователю, но помогают SEO. Что ему нужно?

Подсказка: Это обычно мета-тег description.

Ответ:

-
-
- lang='en'

Шаг 09:

Миссия 9

Расположите шаги настройки метаданных для новой страницы в правильном порядке:

1. Добавить мета-тег charset
2. Указать основной язык с lang
3. Добавить описание страницы для поисковиков
4. Настроить viewport для адаптивности

Подсказка: Сначала базовая кодировка и язык, затем SEO и адаптивность.

Ответ:

- Добавить мета-тег charset
- Указать основной язык с lang
- Добавить описание страницы для поисковиков
- Настроить viewport для адаптивности

Шаг 10:

Миссия 10

Расположите шаги добавления мультязычных метаданных:

1. Создать основной lang для документа
2. Добавить мета-теги для charset
3. Добавить hreflang для альтернативных языков
4. Добавить description для каждого языка

Подсказка: Сначала базовые настройки, затем альтернативные версии.

Ответ:

- Создать основной lang для документа
- Добавить мета-теги для charset
- Добавить hreflang для альтернативных языков
- Добавить description для каждого языка

 Шаг 11:

Миссия 11

Елена хочет, чтобы браузеры правильно понимали направление текста для арабского. Какой атрибут использовать?

Подсказка: Арабский текст читается справа налево.

Ответ:

- **dir='rtl'**
- dir='ltr'
- lang='ar'

 Шаг 12:

Миссия 12

 Сопоставьте метатег с его ролью для локализации.

Подсказка: Некоторые теги помогают браузеру, другие — поисковым системам.

Ответ:

- **charset** — Обеспечивает правильное отображение символов
- **lang** — Помогает определить язык документа
- **viewport** — Настраивает отображение на разных устройствах
- **description** — Помогает поисковикам понять содержание страницы

 Шаг 13:

Миссия 13

Расположите шаги добавления метаданных для многоязычного сайта:

1. Добавить основной lang для документа
2. Добавить charset
3. Добавить hreflang для альтернативных языков
4. Добавить description для каждого языка
5. Настроить viewport

Подсказка: Сначала базовые настройки, затем SEO и адаптивность.

Ответ:

- Добавить основной lang для документа
- Добавить charset
- Добавить hreflang для альтернативных языков
- Добавить description для каждого языка
- Настроить viewport

 Шаг 14:

Миссия 14

Расположите шаги для проверки локализации сайта:

1. Проверить правильное направление текста для каждого языка
2. Проверить корректность отображения символов
3. Проверить description для поисковиков
4. Проверить адаптивность через viewport

Подсказка: Сначала базовые языковые настройки, затем SEO и адаптивность.

Ответ:

- Проверить правильное направление текста для каждого языка
- Проверить корректность отображения символов
- Проверить description для поисковиков
- Проверить адаптивность через viewport

Урок 4 - Многоязычные страницы

Шаг 01:

Миссия 1

Алекс хочет указать язык основной страницы сайта. Какой атрибут HTML используется для этого?

Подсказка: Это помогает браузеру и поисковикам определить язык текста.

Ответ:

- lang='en'
- dir='ltr'
- charset='UTF-8'

Шаг 02:

Миссия 2

Виктория хочет, чтобы поисковые системы знали, что сайт на французском языке. Какой атрибут ей нужен?

Подсказка: Обратите внимание на атрибут, который указывает основной язык документа.

Ответ:

- lang='fr'
- dir='rtl'
- meta name='language'

Шаг 03:

Миссия 3

Иван хочет корректно отображать кириллицу. Какой метатег отвечает за кодировку?

Подсказка: Кодировка обычно указывается в `` с помощью ``.

Ответ:

-
-
-

🔮 Шаг 04:

Миссия 4

Мария хочет, чтобы сайт был адаптивным для мобильных устройств. Какой метатег использовать?

Подсказка: Этот тег обычно содержит width и initial-scale.

Ответ:

-
-
- lang='en'

🔮 Шаг 05:

Миссия 5

📄 Сопоставьте метаданные с их назначением.

Подсказка: Некоторые теги помогают браузеру, другие — поисковикам.

Ответ:

- **lang** — Указывает основной язык страницы
- **charset** — Определяет кодировку текста
- **viewport** — Настраивает отображение на мобильных устройствах
- **description** — Предоставляет описание страницы для поисковых систем

🔮 Шаг 06:

Миссия 6

📄 Сопоставьте метатеги с примером использования.

Подсказка: Обратите внимание на синтаксис и назначение каждого тега.

Ответ:

- **Указать язык** —
- **Кодировка UTF-8** —
- **Описание страницы** —
- **Адаптивность** —

🔮 Шаг 07:

Миссия 7

📄 Сопоставьте задачу с правильным метатегом.

Подсказка: Некоторые теги помогают пользователю, другие — поисковикам.

Ответ:

- **Правильное отображение символов** —
- **Указать язык документа** —
- **Адаптивность** —
- **SEO описание** —

🔮 Шаг 08:

Миссия 8

Дмитрий хочет добавить метаданные для SEO, невидимые пользователю. Что ему нужно?

Подсказка: Обычно это ``.

Ответ:

-
-
- lang='en'

Шаг 09:

Миссия 9

Расположите шаги настройки метаданных для новой страницы:

1. Добавить charset
2. Указать основной lang
3. Добавить description
4. Настроить viewport

Подсказка: Сначала базовая кодировка и язык, затем SEO и адаптивность.

Ответ:

- Добавить charset
- Указать основной lang
- Добавить description
- Настроить viewport

Шаг 10:

Миссия 10

Алексей добавляет метатеги для перевода страницы на несколько языков. Что важно указать в ``?

Подсказка: Атрибут языка помогает браузеру выбрать правильный текст.

Ответ:

- lang='en'
- dir='ltr'
- meta name='viewport'

Шаг 11:

Миссия 11

 Сопоставьте атрибуты и их эффект на многоязычную страницу.

Подсказка: Подумайте, что отвечает за направление текста, а что — за язык.

Ответ:

- lang='ar' — Устанавливает арабский язык страницы
- dir='rtl' — Текст читается справа налево
- lang='en' — Устанавливает английский язык страницы
- dir='ltr' — Текст читается слева направо

Шаг 12:

Миссия 12

 Сопоставьте метаданные с их функцией на странице.

Подсказка: Некоторые теги важны для SEO, другие — для локализации.

Ответ:

- **meta charset** — Определяет кодировку символов
- **meta description** — Описание страницы для поисковых систем
- **lang атрибут** — Указывает язык контента
- **dir атрибут** — Устанавливает направление текста

 Шаг 13:

Миссия 13

Расположите действия при создании многоязычной страницы в правильном порядке:

1. Указать charset
2. Установить основной lang
3. Добавить direction (dir)
4. Добавить метатеги description и viewport

Подсказка: Сначала базовые технические настройки, затем SEO и локализация.

Ответ:

- Указать charset
- Установить основной lang
- Добавить direction (dir)
- Добавить метатеги description и viewport

 Шаг 14:

Миссия 14

Расположите шаги настройки сайта для RTL-языка:

1. Установить lang='ar'
2. Установить dir='rtl'
3. Проверить кодировку UTF-8
4. Добавить описание страницы для SEO

Подсказка: Сначала язык и направление текста, потом кодировка и SEO.

Ответ:

- Установить lang='ar'
- Установить dir='rtl'
- Проверить кодировку UTF-8
- Добавить описание страницы для SEO

 Шаг 15:

Миссия 15

Расположите шаги подготовки многоязычного сайта для тестирования:

1. Добавить все нужные языки и direction
2. Проверить отображение текста на всех языках
3. Настроить SEO метатеги
4. Проверить адаптивность для мобильных устройств

Подсказка: Сначала добавляем контент, затем проверяем корректность отображения и SEO.

Ответ:

- Добавить все нужные языки и direction
- Проверить отображение текста на всех языках
- Настроить SEO метатеги
- Проверить адаптивность для мобильных устройств

 Шаг 16:

Миссия 16

Расположите этапы внедрения локализации на многоязычном сайте:

1. Указать charset и lang
2. Добавить dir для RTL-языков
3. Добавить description и viewport
4. Проверить отображение всех элементов на разных языках

Подсказка: Локализация начинается с базовых тегов, затем направленность текста, SEO и проверка отображения.

Ответ:

- Указать charset и lang
- Добавить dir для RTL-языков
- Добавить description и viewport
- Проверить отображение всех элементов на разных языках

 Модуль 5 - Мультимедиа и интеграции

 Урок 1 - Атрибуты audio и video

 Шаг 01:

Миссия 1

Мария хочет, чтобы видео автоматически запускалось при загрузке страницы. Какой атрибут нужно использовать?

Подсказка: Это свойство включает воспроизведение без клика.

Ответ:

- **autoplay**
- controls
- loop

 Шаг 02:

Миссия 2

 Сопоставьте атрибуты аудио/видео с их функцией.

Подсказка: Подумайте, какой атрибут влияет на управление, воспроизведение и повторение.

Ответ:

- **controls** — Отображает элементы управления воспроизведением
- **autoplay** — Автоматически начинает воспроизведение
- **loop** — Повторяет воспроизведение после окончания
- **muted** — Отключает звук

🔗 Шаг 03:

Миссия 3

Петр хочет, чтобы видео повторялось бесконечно. Какой атрибут ему нужен?

Подсказка: Атрибут связан с циклическим воспроизведением.

Ответ:

- **loop**
- **autoplay**

🔗 Шаг 04:

Миссия 4

Аня добавляет аудиофайл, который должен начинать воспроизведение без звука. Какой атрибут использовать?

Подсказка: Автозапуск и отсутствие звука часто идут вместе.

Ответ:

- **muted autoplay**
- **controls**

🔗 Шаг 05:

Миссия 5

📄 Сопоставьте атрибуты с описанием поведения мультимедиа.

Подсказка: Подумайте, какие атрибуты включают элементы управления или звук.

Ответ:

- **preload='auto'** — Браузер загружает весь файл заранее
- **preload='metadata'** — Загружаются только метаданные
- **controls** — Показывает кнопки воспроизведения и паузы
- **muted** — Отключает звук

🔗 Шаг 06:

Миссия 6

📄 Сопоставьте форматы мультимедиа с их поддержкой в браузерах.

Подсказка: Некоторые форматы более универсальны.

Ответ:

- **MP3** — Поддерживается почти всеми браузерами
- **OGG** — Поддерживается большинством, но не Safari
- **MP4** — Поддерживается всеми современными браузерами
- **WEBM** — Хорошо работает в Chrome и Firefox

🔗 Шаг 07:

Миссия 7

Расположите шаги добавления видео с локального файла:

1. Добавить тег
2. Указать src и type
3. Добавить controls и другие атрибуты

4. Проверить воспроизведение в разных браузерах

Подсказка: Сначала структура, потом атрибуты, затем проверка.

Ответ:

- Добавить тег
- Указать src и type
- Добавить controls и другие атрибуты
- Проверить воспроизведение в разных браузерах

Шаг 08:

Миссия 8

Коля хочет, чтобы пользователь видел кнопки управления видео. Какой атрибут добавить?

Подсказка: Без него элемент управления не будет видимым.

Ответ:

- **controls**
- autoplay

Шаг 09:

Миссия 9

 Сопоставьте атрибуты с эффектом на аудио/видео.

Подсказка: Подумайте, что делает каждое свойство.

Ответ:

- **preload='none'** — Не загружает файл до воспроизведения
- **loop** — Повторяет воспроизведение
- **muted** — Отключает звук
- **autoplay** — Запускает воспроизведение автоматически

Шаг 10:

Миссия 10

 Сопоставьте форматы видео с их браузерной поддержкой.

Подсказка: Некоторые форматы универсальные, другие — ограничены.

Ответ:

- **MP4** — Поддерживается всеми современными браузерами
- **WEBM** — Поддержка в Chrome и Firefox
- **OGG** — Поддержка в большинстве браузеров, кроме Safari
- **AVI** — Старый формат, мало поддерживается

Шаг 11:

Миссия 11

Расположите шаги добавления видео с YouTube:

Ответ:

- Добавить тег `</li>` `<li>` Указать src на видео `</li>` `<li>` Задать ширину и высоту `</li>` `<li>` Проверить отображение на странице `</li>` `</ul>` `</details>`

Урок 2 - Постеры и preload для видео

Шаг 01:

Миссия 1

Иван добавляет видео на страницу и хочет, чтобы при загрузке отображалось превью, а не первый кадр видео. Какой атрибут использовать?

Подсказка: Этот атрибут задаёт изображение-постер до начала воспроизведения.

Ответ:

- **poster**
- preload
- autoplay

Шаг 02:

Миссия 2

Мария хочет, чтобы видео заранее подгружалось, чтобы избежать задержек при воспроизведении. Какой атрибут использовать?

Подсказка: Подумайте, какой атрибут управляет предварительной загрузкой видео.

Ответ:

- **preload**
- poster
- controls

Шаг 03:

Миссия 3

 Сопоставьте атрибуты видео с их функцией.

Подсказка: Подумайте, что управляет загрузкой, отображением и автозапуском видео.

Ответ:

- **poster** — Отображает изображение до начала воспроизведения
- **preload='auto'** — Загружает видео полностью при загрузке страницы
- **preload='metadata'** — Загружает только метаданные видео
- **preload='none'** — Не подгружает видео до воспроизведения

Шаг 04:

Миссия 4

 Сопоставьте значения preload с результатом в браузере.

Подсказка: Подумайте, что загружается заранее.

Ответ:

- **auto** — Браузер загружает весь файл
- **metadata** — Браузер загружает только информацию о длительности и размере
- **none** — Браузер не загружает видео до воспроизведения

Шаг 05:

Миссия 5

Петр хочет, чтобы видео показывало изображение до начала воспроизведения и одновременно заранее подгружалось. Какие атрибуты использовать?

Подсказка: Вам нужны атрибут для постера и для предварительной загрузки.

Ответ:

- **poster**
- **preload**
- controls

Шаг 06:

Миссия 6

Аня хочет, чтобы видео отображалось без загрузки полного файла сразу. Какой preload выбрать?

Подсказка: Видео не должно загружаться до воспроизведения.

Ответ:

- **none**
- auto
- metadata

Шаг 07:

Миссия 7

 Сопоставьте атрибуты с их эффектом для видео на странице.

Подсказка: Подумайте, какой атрибут управляет тем, что видит пользователь до старта.

Ответ:

- **poster** — Изображение, видимое до начала воспроизведения
- **controls** — Отображает кнопки воспроизведения и паузы
- **autoplay** — Автоматически запускает воспроизведение
- **loop** — Повторяет видео после окончания

Шаг 08:

Миссия 8

 Сопоставьте значения preload с их особенностями.

Подсказка: Подумайте, что подгружается полностью, частично или не подгружается.

Ответ:

- **auto** — Полная загрузка видео при загрузке страницы
- **metadata** — Загрузка только информации о длительности и размере
- **none** — Видео не загружается до воспроизведения

Шаг 09:

Миссия 9

Кто-то хочет добавить превью к видео, но оставить автозапуск. Что добавить?

Подсказка: Постер работает независимо от autoplay.

Ответ:

- **poster**
- controls

🔔 Шаг 10:

Миссия 10

Расположите шаги добавления видео с постером и preload:

1. Добавить тег <video>
2. Указать src
3. Добавить poster и preload
4. Проверить отображение постера и воспроизведение

Подсказка: Сначала структура, затем атрибуты, потом проверка.

Ответ:

- Добавить тег <video>
- Указать src
- Добавить poster и preload
- Проверить отображение постера и воспроизведение

🔔 Шаг 11:

Миссия 11

Расположите шаги добавления видео с автозапуском и постером:

1. Добавить тег <video>
2. Указать src
3. Добавить poster и autoplay
4. Проверить воспроизведение и видимость постера

Подсказка: Сначала тег и источник, потом атрибуты, затем тестирование.

Ответ:

- Добавить тег <video>
- Указать src
- Добавить poster и autoplay
- Проверить воспроизведение и видимость постера

🔔 Шаг 12:

Миссия 12

Коля хочет, чтобы видео не загружалось полностью до воспроизведения. Какой preload выбрать?

Подсказка: Нужно минимальное использование трафика.

Ответ:

- none
- auto
- metadata

🔔 Шаг 13:

Миссия 13

📄 Сопоставьте атрибуты с их эффектом для видео с постером.

Подсказка: Подумайте, что влияет на визуальное отображение и загрузку видео.

Ответ:

- **poster** — Изображение, видимое до начала воспроизведения
- **preload='auto'** — Видео загружается полностью
- **preload='metadata'** — Загружаются только метаданные
- **preload='none'** — Видео не загружается до воспроизведения

🔗 Шаг 14:

Миссия 14

📄 Сопоставьте типы postera с применением.

Подсказка: Разные форматы изображений могут использоваться для превью.

Ответ:

- **JPEG** — Для обычных фото постеров
- **PNG** — Для прозрачных элементов постера
- **GIF** — Для анимации в постере
- **SVG** — Для векторного постера

🔗 Шаг 15:

Миссия 15

Расположите шаги оптимизации загрузки видео с постером:

1. Добавить тег `<video>`
2. Указать `src` и `poster`
3. Выбрать `preload='metadata'` для экономии трафика
4. Проверить отображение постера и воспроизведение

Подсказка: Сначала структура и источник, потом оптимизация, затем проверка.

Ответ:

- Добавить тег `<video>`
- Указать `src` и `poster`
- Выбрать `preload='metadata'` для экономии трафика
- Проверить отображение постера и воспроизведение

🔗 Шаг 16:

Миссия 16

Расположите шаги добавления видео с постером, автозапуском и `mute`:

1. Добавить тег `<video>`
2. Указать `src`
3. Добавить `poster`, `autoplay` и `muted`
4. Проверить воспроизведение и видимость постера

Подсказка: Сначала структура, потом атрибуты, затем тестирование.

Ответ:

- Добавить тег `<video>`
- Указать `src`
- Добавить `poster`, `autoplay` и `muted`
- Проверить воспроизведение и видимость постера

Урок 3 - Субтитры через track

Шаг 01:

Миссия 1

Анна хочет добавить субтитры к видео. Какой тег использовать внутри `<video>`?

Подсказка: Этот тег отвечает за отдельные дорожки текста для видео.

Ответ:

- **track**
- caption
- subtitle

Шаг 02:

Миссия 2

Иван хочет, чтобы субтитры отображались автоматически при воспроизведении видео. Какой атрибут нужно добавить к `<track>`?

Подсказка: Атрибут включает субтитры по умолчанию.

Ответ:

- **default**
- autoplay
- visible

Шаг 03:

Миссия 3

 Сопоставьте атрибуты `<track>` с их функцией.

Подсказка: Каждый атрибут определяет язык, тип или отображение субтитров.

Ответ:

- **kind** — Определяет тип дорожки (subtitles, captions, descriptions)
- **src** — Путь к файлу с субтитрами
- **srclang** — Язык субтитров
- **label** — Название дорожки для выбора пользователем
- **default** — Субтитры отображаются по умолчанию

Шаг 04:

Миссия 4

 Сопоставьте типы дорожек с их назначением.

Подсказка: Подумайте, какие дорожки предназначены для зрителей с ограничениями слуха или для перевода.

Ответ:

- **subtitles** — Перевод речи видео на другой язык
- **captions** — Субтитры для людей с нарушениями слуха
- **descriptions** — Аудиоописание для слабовидящих
- **chapters** — Разделение видео на главы
- **metadata** — Дополнительная информация о видео

Шаг 05:

Миссия 5

Олег хочет, чтобы пользователь мог выбирать между английскими и русскими субтитрами. Что нужно указать?

Подсказка: Нужно задать язык и метку для каждой дорожки.

Ответ:

- **srclang**
- **label**
- **autoplay**

Шаг 06:

Миссия 6

Настя хочет скрыть субтитры по умолчанию. Какой атрибут нужно не использовать?

Подсказка: Атрибут по умолчанию делает субтитры видимыми сразу.

Ответ:

- **default**
- **srclang**

Шаг 07:

Миссия 7

 Сопоставьте элементы `<track>` с их описанием.

Подсказка: Подумайте, что важно для доступности и локализации видео.

Ответ:

- **label** — Название дорожки для выбора пользователем
- **srclang** — Язык субтитров
- **kind** — Тип субтитров (captions, subtitles)
- **src** — Файл с субтитрами
- **default** — Отображение по умолчанию

Шаг 08:

Миссия 8

 Сопоставьте типы дорожек с примерами использования.

Подсказка: Подумайте, для кого предназначены разные типы дорожек.

Ответ:

- **subtitles** — Перевод речи видео на другой язык
- **captions** — Субтитры для слабослышащих
- **descriptions** — Аудиоописание для слабовидящих
- **chapters** — Навигация по видео
- **metadata** — Информация о видео

Шаг 09:

Миссия 9

Саша хочет добавить несколько субтитров на английском, русском и испанском. Какие атрибуты обязательны для каждой дорожки?

Подсказка: Нужно указать путь к файлу, язык и метку.

Ответ:

- **src**
- **srclang**
- **label**

Шаг 10:

Миссия 10

Расположите шаги добавления субтитров к видео в правильном порядке:

1. Добавить тег `<video>`
2. Создать файлы субтитров (.vtt)
3. Добавить теги `<track>` с атрибутами `src`, `srclang`, `label`
4. Опционально добавить `default` для одной дорожки
5. Проверить отображение и работу субтитров

Подсказка: Сначала структура, затем файлы, потом теги, настройка отображения и тестирование.

Ответ:

- Добавить тег `<video>`
- Создать файлы субтитров (.vtt)
- Добавить теги `<track>` с атрибутами `src`, `srclang`, `label`
- Опционально добавить `default` для одной дорожки
- Проверить отображение и работу субтитров

Шаг 11:

Миссия 11

Расположите шаги добавления нескольких языковых субтитров:

1. Создать файлы субтитров для каждого языка
2. Добавить тег `<video>`
3. Добавить теги `<track>` с правильными атрибутами
4. Проверить работу переключения языков

Подсказка: Сначала файлы, потом структура, затем теги и тестирование.

Ответ:

- Создать файлы субтитров для каждого языка
- Добавить тег `<video>`
- Добавить теги `<track>` с правильными атрибутами
- Проверить работу переключения языков

Шаг 12:

Миссия 12

Петя хочет, чтобы субтитры отображались только для слабослышащих пользователей. Какой `kind` выбрать?

Подсказка: Этот тип предназначен для accessibility.

Ответ:

- **captions**

- subtitles

🔮 Шаг 13:

Миссия 13

📄 Сопоставьте атрибуты <track> с их особенностями для разных случаев:

Подсказка: Подумайте, какой атрибут нужен для языка, видимости и типа субтитров.

Ответ:

- **srclang** — Указывает язык субтитров
- **label** — Отображаемое имя дорожки
- **kind** — Тип субтитров (captions, subtitles)
- **src** — Ссылка на файл субтитров
- **default** — Показывать субтитры по умолчанию

🔮 Шаг 14:

Миссия 14

📄 Сопоставьте форматы файлов субтитров с поддержкой браузеров:

Подсказка: Разные браузеры лучше работают с разными форматами.

Ответ:

- **.vtt** — Стандарт для современных браузеров
- **.srt** — Старый формат, иногда требует конвертации
- **.sbv** — Формат Google
- **.dfxp** — Для видео на платформе iTunes

🔮 Шаг 15:

Миссия 15

Расположите шаги добавления субтитров с проверкой совместимости:

1. Создать файлы субтитров в .vtt
2. Добавить тег <video>
3. Добавить теги <track> с атрибутами
4. Проверить работу в разных браузерах

Подсказка: Сначала файлы, потом структура и теги, затем тестирование.

Ответ:

- Создать файлы субтитров в .vtt
- Добавить тег <video>
- Добавить теги <track> с атрибутами
- Проверить работу в разных браузерах

🔮 Шаг 16:

Миссия 16

Расположите шаги добавления нескольких типов дорожек и проверки отображения:

1. Создать файлы captions, subtitles и descriptions
2. Добавить тег <video>
3. Добавить теги <track> с правильными атрибутами для каждого файла
4. Проверить отображение всех дорожек и переключение между ними

Подсказка: Сначала файлы, потом структура и теги, затем тестирование.

Ответ:

- Создать файлы captions, subtitles и descriptions
- Добавить тег <video>
- Добавить теги <track> с правильными атрибутами для каждого файла
- Проверить отображение всех дорожек и переключение между ними

 **Шаг 17:**

Миссия 17

Маша хочет сделать одну дорожку субтитров видимой по умолчанию, остальные доступны для выбора. Что добавить?

Подсказка: Один тег track может иметь специальный атрибут для этого.

Ответ:

- **default**
- autoplay

Урок 4 - Встраивание карт и видео через iframe

 **Шаг 01:**

Миссия 1

Анна хочет встроить карту на страницу. Какой тег использовать?

Подсказка: Этот тег позволяет вставлять внешние ресурсы внутри HTML.

Ответ:

- **iframe**
- embed
- object

 **Шаг 02:**

Миссия 2

Иван хочет встроить видео с YouTube. Какой атрибут нужен, чтобы видео автоматически масштабировалось по размеру контейнера?

Подсказка: Атрибут связан с шириной и высотой.

Ответ:

- **width и height**
- src
- allowfullscreen

 **Шаг 03:**

Миссия 3

Олег хочет разрешить пользователю полноэкранный режим для встроеного видео. Какой атрибут добавить?

Подсказка: Это специальный флаг для видео в iframe.

Ответ:

- **allowfullscreen**
- autoplay

🔗 Шаг 04:

Миссия 4

📄 Сопоставьте атрибуты iframe с их назначением.

Подсказка: Подумайте, какой атрибут управляет размером, источником или функциональностью.

Ответ:

- **src** — URL внешнего ресурса, который нужно встроить
- **width** — Ширина iframe
- **height** — Высота iframe
- **allowfullscreen** — Разрешает полноэкранный режим
- **loading** — Отложенная загрузка контента

🔗 Шаг 05:

Миссия 5

Настя хочет, чтобы карта загружалась только при скролле к ней. Какой атрибут использовать?

Подсказка: Это механизм ленивой загрузки.

Ответ:

- **loading="lazy"**
- defer

🔗 Шаг 06:

Миссия 6

Петя хочет запретить iframe менять размеры или содержимое родительской страницы. Какой атрибут использовать?

Подсказка: Это часть политики безопасности.

Ответ:

- **sandbox**
- allowfullscreen

🔗 Шаг 07:

Миссия 7

📄 Сопоставьте атрибуты iframe с их типичной функцией.

Подсказка: Думайте, что нужно для безопасности, размера и загрузки.

Ответ:

- **sandbox** — Ограничивает действия встроенного контента
- **width** — Ширина окна iframe
- **height** — Высота окна iframe
- **src** — Ссылка на внешнюю страницу
- **loading** — Отложенная загрузка для оптимизации

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте типы встроенного контента с примерами.

Подсказка: Вспомните, что чаще всего встраивают через iframe.

Ответ:

- **Видео** — YouTube
- **Карты** — Google Maps
- **Документы** — PDF
- **Веб-приложения** — Интерактивные формы
- **Графика** — SVG или изображения

Шаг 09:

Миссия 9

Оля хочет, чтобы встроенное видео автоматически воспроизводилось. Какой атрибут добавить в src URL?

Подсказка: Этот параметр часто используется для YouTube.

Ответ:

- **autoplay=1**
- loop=1

Шаг 10:

Миссия 10

Расположите шаги для встраивания карты через iframe:

1. Создать HTML контейнер
2. Скопировать ссылку на карту
3. Добавить тег <iframe> с атрибутами src, width, height
4. Добавить дополнительные атрибуты (loading, sandbox)
5. Проверить отображение карты на странице

Подсказка: Сначала подготовка, потом вставка, затем оптимизация и проверка.

Ответ:

- Создать HTML контейнер
- Скопировать ссылку на карту
- Добавить тег <iframe> с атрибутами src, width, height
- Добавить дополнительные атрибуты (loading, sandbox)
- Проверить отображение карты на странице

Шаг 11:

Миссия 11

Расположите шаги для безопасного встраивания видео через iframe:

1. Добавить тег <iframe>
2. Указать src на видео
3. Добавить sandbox для ограничения действий
4. Добавить allowfullscreen для опции полноэкранного режима
5. Проверить работу видео на странице

Подсказка: Сначала структура, потом ограничения и проверки.

Ответ:

- Добавить тег <iframe>
- Указать src на видео

- Добавить `sandbox` для ограничения действий
- Добавить `allowfullscreen` для опции полноэкранного режима
- Проверить работу видео на странице

Шаг 12:

Миссия 12

Саша хочет, чтобы `iframe` подстраивался под мобильные устройства. Как это сделать?

Подсказка: Используйте CSS или атрибуты для адаптивного размера.

Ответ:

- `width="100%"`
- `height="auto"`
- `fixed-size`

Шаг 13:

Миссия 13

 Сопоставьте атрибуты `iframe` с их особенностями:

Подсказка: Думайте о безопасности, производительности и отображении.

Ответ:

- **`loading`** — Отложенная загрузка контента для ускорения рендеринга страницы
- **`sandbox`** — Ограничивает действия встроенного контента
- **`allowfullscreen`** — Разрешает видео открываться на полный экран
- **`width`** — Ширина `iframe`
- **`height`** — Высота `iframe`

Модуль 6 - Метаданные и SEO

Урок 1 - Теги `title` и `description`

Шаг 01:

Миссия 1

Анна хочет встроить карту на страницу. Какой тег использовать?

Подсказка: Этот тег позволяет вставлять внешние ресурсы внутри HTML.

Ответ:

- **`iframe`**
- `embed`
- `object`

Шаг 02:

Миссия 2

Иван хочет встроить видео с YouTube. Какой атрибут нужен, чтобы видео автоматически масштабировалось по размеру контейнера?

Подсказка: Атрибут связан с шириной и высотой.

Ответ:

- **`width` и `height`**
- `src`

- `allowfullscreen`

🔧 Шаг 03:

Миссия 3

Олег хочет разрешить пользователю полноэкранный режим для встроенного видео. Какой атрибут добавить?

Подсказка: Это специальный флаг для видео в `iframe`.

Ответ:

- `allowfullscreen`
- `autoplay`

🔧 Шаг 04:

Миссия 4

📄 Сопоставьте атрибуты `iframe` с их назначением.

Подсказка: Подумайте, какой атрибут управляет размером, источником или функциональностью.

Ответ:

- `src` — Ссылка на внешний ресурс
- `width` — Ширина `iframe`
- `height` — Высота `iframe`
- `allowfullscreen` — Разрешает полноэкранный режим
- `loading` — Отложенная загрузка контента

🔧 Шаг 05:

Миссия 5

Настя хочет, чтобы карта загружалась только при скролле к ней. Какой атрибут использовать?

Подсказка: Это механизм ленивой загрузки.

Ответ:

- `loading="lazy"`
- `defer`

🔧 Шаг 06:

Миссия 6

Петя хочет запретить `iframe` менять размеры или содержимое родительской страницы. Какой атрибут использовать?

Подсказка: Это часть политики безопасности.

Ответ:

- `sandbox`
- `allowfullscreen`

🔧 Шаг 07:

Миссия 7

📄 Сопоставьте атрибуты `iframe` с их типичной функцией.

Подсказка: Думайте о безопасности, размере и загрузке.

Ответ:

- **sandbox** — Ограничивает действия встроенного контента
- **width** — Ширина iframe
- **height** — Высота iframe
- **src** — Ссылка на внешнюю страницу
- **loading** — Отложенная загрузка для оптимизации

💡 **Шаг 08:**

Миссия 8

📄 Сопоставьте типы встроенного контента с примерами.

Подсказка: Вспомните, что чаще всего встраивают через iframe.

Ответ:

- **Видео** — YouTube
- **Карты** — Google Maps
- **Документы** — PDF
- **Веб-приложения** — Интерактивные формы
- **Графика** — SVG или изображения

💡 **Шаг 09:**

Миссия 9

Оля хочет, чтобы встроенное видео автоматически воспроизводилось. Какой параметр добавить в URL?

Подсказка: Часто используется для YouTube.

Ответ:

- **autoplay=1**
- **loop=1**

📖 Урок 2 - Open Graph и соцсети

💡 **Шаг 01:**

Миссия 1

Маша хочет, чтобы при публикации ссылки в соцсетях отображалась картинка и описание страницы. Какой тег использовать?

Подсказка: Эти метатеги помогают соцсетям правильно показывать превью.

Ответ:

- **meta property="og:image"**
- **link rel="stylesheet"**
- **script src**

💡 **Шаг 02:**

Миссия 2

Петя хочет, чтобы заголовок страницы корректно отображался при шаринге в Facebook. Какой тег использовать?

Подсказка: Заголовок в Open Graph отдельный от обычного <title>.

Ответ:

- **meta property="og:title"**
- meta name="description"
- meta charset

Шаг 03:

Миссия 3

Оля хочет добавить описание страницы для превью в соцсетях. Какой тег выбрать?

Подсказка: Это короткий текст, который объясняет содержимое страницы.

Ответ:

- **meta property="og:description"**
- meta property="og:url"
- link rel="icon"

Шаг 04:

Миссия 4

 Сопоставьте Open Graph теги с их назначением.

Подсказка: Подумайте, какой тег отвечает за изображение, заголовок, описание или URL.

Ответ:

- **og:title** — Заголовок страницы для соцсетей
- **og:description** — Краткое описание страницы
- **og:image** — Изображение для превью
- **og:url** — URL страницы
- **og:type** — Тип контента (article, website и т.д.)

Шаг 05:

Миссия 5

Саша хочет, чтобы ссылка на статью корректно отображалась в Telegram. Какой тег важен для превью?

Подсказка: Telegram поддерживает Open Graph для отображения изображений и заголовков.

Ответ:

- **og:image**
- meta charset

Шаг 06:

Миссия 6

Игорь хочет, чтобы при шаринге страницы автоматически подтягивался правильный URL. Какой тег использовать?

Подсказка: Это помогает соцсетям точно знать, какая страница должна открыться.

Ответ:

- **og:url**
- og:type

🔮 Шаг 07:

Миссия 7

📄 Сопоставьте элементы Open Graph с примером содержимого.

Подсказка: Подумайте, какой тег будет содержать текст, изображение или URL.

Ответ:

- **og:title** — "Лучшие советы для веб-разработчиков"
- **og:description** — "Собрание полезных материалов по HTML, CSS и JS"
- **og:image** — <https://site.com/images/preview.jpg>
- **og:url** — <https://site.com/blog/tips>
- **og:type** — article

🔮 Шаг 08:

Миссия 8

📄 Сопоставьте метатеги Twitter с их назначением.

Подсказка: Некоторые соцсети требуют специальных тегов для правильного отображения.

Ответ:

- **twitter:title** — Заголовок для твита
- **twitter:description** — Описание для твита
- **twitter:image** — Изображение для твита
- **twitter:card** — Тип карты (summary, summary_large_image)

🔮 Шаг 09:

Миссия 9

Лена хочет, чтобы у статьи был тип контента "article" для Open Graph. Какой тег добавить?

Подсказка: Это помогает соцсетям понимать структуру страницы.

Ответ:

- **meta property="og:type" content="article"**
- **meta name="keywords"**

🔮 Шаг 10:

Миссия 10

Виталий хочет, чтобы ссылка в соцсетях подтягивала изображение с определенной высотой и шириной. Какой тег важен?

Подсказка: Open Graph позволяет указывать размеры изображения.

Ответ:

- **og:image:width** и **og:image:height**
- **og:locale**

🔮 Шаг 11:

Миссия 11

Расположите шаги добавления Open Graph тегов для страницы блога в правильном порядке:

1. Определить заголовок и описание страницы
2. Выбрать изображение для превью

3. Добавить метатеги `og:title`, `og:description`, `og:image`, `og:url`
4. Указать тип контента через `og:type`
5. Проверить корректность отображения в соцсетях

Подсказка: Сначала контент, потом теги, затем проверка.

Ответ:

- Определить заголовок и описание страницы
- Выбрать изображение для превью
- Добавить метатеги `og:title`, `og:description`, `og:image`, `og:url`
- Указать тип контента через `og:type`
- Проверить корректность отображения в соцсетях

Шаг 12:

Миссия 12

Расположите шаги добавления Twitter Cards на страницу:

1. Определить заголовок, описание и изображение
2. Выбрать тип карты (`summary` или `summary_large_image`)
3. Добавить метатеги `twitter:title`, `twitter:description`, `twitter:image`, `twitter:card`
4. Проверить отображение через Twitter Card Validator

Подсказка: Сначала собираем контент, потом добавляем теги, затем проверяем.

Ответ:

- Определить заголовок, описание и изображение
- Выбрать тип карты (`summary` или `summary_large_image`)
- Добавить метатеги `twitter:title`, `twitter:description`, `twitter:image`, `twitter:card`
- Проверить отображение через Twitter Card Validator

Шаг 13:

Миссия 13

Никита хочет, чтобы Open Graph теги корректно работали при шаринге на всех платформах. Что важно проверить?

Подсказка: Тестирование через социальные отладчики.

Ответ:

- Проверить через Facebook Debugger
- Проверить через Twitter Card Validator
- Просто обновить страницу

Шаг 14:

Миссия 14

 Сопоставьте теги Open Graph с их примерами для статьи.

Подсказка: Заголовок, описание и изображение должны быть уникальными.

Ответ:

- **og:title** — "Как создавать адаптивные сайты"
- **og:description** — "Полное руководство по адаптивной верстке"
- **og:image** — <https://site.com/images/adaptive.png>
- **og:url** — <https://site.com/blog/adaptive>

- **og:type** — article

Урок 3 - Favicon и иконки сайта

Шаг 01:

Миссия 1

Аня хочет, чтобы на вкладке браузера отображалась иконка сайта. Какой тег использовать?

Подсказка: Это стандартный тег для подключения favicon.

Ответ:

- **link rel="icon" href="favicon.ico"**
- meta name="icon"
- script src="favicon.ico"

Шаг 02:

Миссия 2

Борис хочет подключить иконку PNG формата для браузеров с высокой плотностью пикселей. Какой атрибут важен?

Подсказка: Используется для определения размеров иконки.

Ответ:

- **sizes="32x32"**
- type="image/jpeg"
- rel="stylesheet"

Шаг 03:

Миссия 3

Катя хочет добавить иконку для Apple Touch устройств. Какой тег использовать?

Подсказка: Этот тег отличается от стандартного favicon для браузеров.

Ответ:

- **link rel="apple-touch-icon" href="apple-touch-icon.png"**
- link rel="icon" href="favicon.ico"
- meta name="apple-icon"

Шаг 04:

Миссия 4

 Сопоставьте тип иконки с её назначением.

Подсказка: Подумайте, где каждая иконка будет отображаться.

Ответ:

- **favicon.ico** — Иконка для вкладки браузера
- **apple-touch-icon.png** — Иконка для устройств Apple при добавлении на экран
- **favicon-32x32.png** — Иконка для браузеров с высокой плотностью пикселей
- **favicon-16x16.png** — Стандартная маленькая иконка для большинства браузеров
- **mstile-150x150.png** — Иконка для Windows плиток

🔗 Шаг 05:

Миссия 5

📖 Сопоставьте атрибуты link с их назначением для иконок.

Подсказка: Подумайте, какой атрибут отвечает за путь, размеры и тип.

Ответ:

- **rel** — Определяет тип иконки (icon, apple-touch-icon)
- **href** — Указывает путь к файлу иконки
- **sizes** — Задаёт размеры иконки
- **type** — Указывает MIME-тип изображения

🔗 Шаг 06:

Миссия 6

Дима хочет, чтобы браузеры автоматически использовали подходящую иконку для разных устройств. Что важно добавить?

Подсказка: Указываем несколько размеров и форматов.

Ответ:

- **Несколько тегов link с разными sizes**
- **Файлы разных форматов (.ico, .png)**
- Оставить только один favicon.ico

🔗 Шаг 07:

Миссия 7

Света заметила, что старые браузеры не показывают PNG иконки. Что использовать для поддержки?

Подсказка: Старые браузеры требуют специфический формат.

Ответ:

- **favicon.ico**
- apple-touch-icon.png

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте размеры иконок с устройствами или случаями использования.

Подсказка: Размеры важны для правильного отображения на разных устройствах.

Ответ:

- **16x16** — Мини-иконка для вкладок браузера
- **32x32** — Стандартная иконка для браузеров с высокой плотностью
- **48x48** — Windows XP и старые системы
- **150x150** — Плитки Windows 8 и 10
- **180x180** — Apple Touch и Retina устройства

🔗 Шаг 09:

Миссия 9

Игорь хочет подключить favicon через HTML5. Какой формат файла лучше использовать?

Подсказка: HTML5 поддерживает больше форматов, чем старые версии.

Ответ:

- **.png**
- **.bmp**

 **Шаг 10:**

Миссия 10

Олег хочет, чтобы иконка сайта открывалась корректно при сохранении страницы на рабочий стол. Что добавить?

Подсказка: Для мобильных устройств используется специальный тег.

Ответ:

- **link rel="apple-touch-icon"**
- **link rel="shortcut icon"**

 **Шаг 11:**

Миссия 11

 Сопоставьте типы иконок с браузерами или устройствами.

Подсказка: Разные браузеры читают разные форматы и теги.

Ответ:

- **favicon.ico** — Все браузеры
- **favicon-32x32.png** — Современные браузеры с высокой плотностью пикселей
- **apple-touch-icon.png** — iPhone и iPad
- **mstile-150x150.png** — Windows плитки

 **Шаг 12:**

Миссия 12

Расположите шаги подключения иконок сайта в правильном порядке:

1. Создать иконки разных размеров
2. Добавить теги **link rel="icon"** и **rel="apple-touch-icon"**
3. Указать атрибут **sizes** для PNG иконок
4. Проверить отображение в браузерах и на мобильных устройствах

Подсказка: Сначала создаем файлы, потом подключаем, затем проверяем.

Ответ:

- Создать иконки разных размеров
- Добавить теги **link rel="icon"** и **rel="apple-touch-icon"**
- Указать атрибут **sizes** для PNG иконок
- Проверить отображение в браузерах и на мобильных устройствах

 **Шаг 13:**

Миссия 13

Расположите шаги оптимизации иконок для всех устройств:

1. Сделать **favicon.ico** для старых браузеров
2. Добавить PNG иконки для современных браузеров
3. Добавить Apple Touch иконки

4. Добавить плитки Windows (mstile)
5. Проверить корректность на всех платформах

Подсказка: Сначала базовые форматы, потом специальные для разных платформ.

Ответ:

- Сделать favicon.ico для старых браузеров
- Добавить PNG иконки для современных браузеров
- Добавить Apple Touch иконки
- Добавить плитки Windows (mstile)
- Проверить корректность на всех платформах

Урок 4 - Проверка SEO в Lighthouse

Шаг 01:

Миссия 1

Алекс хочет проверить SEO своего сайта. Какой инструмент встроен в Chrome для анализа SEO?

Подсказка: Инструмент показывает оценку по различным метрикам сайта.

Ответ:

- **Lighthouse**
- Webpack
- Postman

Шаг 02:

Миссия 2

Мария открыла Lighthouse. Какой раздел показывает метаданные и заголовки страниц?

Подсказка: Этот раздел помогает проверить SEO и соцсети.

Ответ:

- **SEO**
- Performance
- Accessibility

Шаг 03:

Миссия 3

Павел заметил предупреждение в Lighthouse: "Документ не имеет title". Что это значит?

Подсказка: Этот тег помогает поисковым системам понять содержимое страницы.

Ответ:

- **В документе отсутствует тег**
- Не подключен favicon

Шаг 04:

Миссия 4

 Сопоставьте элементы страницы с проверками SEO в Lighthouse.

Подсказка: Подумайте, какая проверка относится к какому элементу.

Ответ:

- **title** — Должен быть уникальным для каждой страницы

- **meta description** — Краткое описание страницы для поисковиков
- **alt для изображений** — Помогает поисковикам понять содержание изображений
- **h1** — Главный заголовок страницы
- **robots.txt** — Контролирует индексирование страниц поисковыми системами

🔮 Шаг 05:

Миссия 5

Алина заметила, что Lighthouse показывает низкую оценку SEO из-за отсутствия заголовков h2. Что это значит?

Подсказка: Заголовки помогают структурировать контент.

Ответ:

- **Страница плохо структурирована для поисковиков**
- Неправильный favicon подключен

🔮 Шаг 06:

Миссия 6

Вадим хочет проверить мобильную версию сайта через Lighthouse. Какой параметр важен?

Подсказка: Смотрите на адаптивность и viewport.

Ответ:

- **Mobile Friendly**
- Desktop Only

🔮 Шаг 07:

Миссия 7

📄 Сопоставьте типы ошибок SEO с последствиями.

Подсказка: Подумайте, как ошибки влияют на поисковые системы.

Ответ:

- **Отсутствие title** — Низкая видимость в поиске
- **Дублирование meta description** — Плохая уникальность контента
- **Нет alt на изображениях** — Поисковики не понимают изображения
- **Ошибки 404** — Страницы не индексируются
- **Медленная загрузка** — Понижает рейтинг страницы

🔮 Шаг 08:

Миссия 8

Наташа видит предупреждение Lighthouse: "Document has no meta description". Что делать?

Подсказка: Это важно для отображения сниппетов в поиске.

Ответ:

- **Добавить тег meta name="description"**
- Удалить favicon

🔮 Шаг 09:

Миссия 9

Сергей хочет оценить SEO после внесения изменений. Что нужно сделать в Lighthouse?

Подсказка: Необходимо обновить отчет после правок.

Ответ:

- **Перезапустить аудит**
- Закрыть вкладку

Модуль 7 - ♥ Доступность (A11y)

Урок 1 - Атрибуты alt и aria-

Шаг 01:

Миссия 1

Анна добавляет изображение на сайт. Какой атрибут нужен, чтобы описание изображения было доступно для людей с нарушениями зрения?

Подсказка: Этот атрибут помогает экранным читалкам.

Ответ:

- **alt**
- title
- src

Шаг 02:

Миссия 2

Иван хочет сделать кнопку более доступной. Какой атрибут aria следует использовать для обозначения её роли?

Подсказка: Атрибут начинается с "aria-" и описывает назначение элемента.

Ответ:

- **aria-label**
- alt
- title

Шаг 03:

Миссия 3

Ольга заметила, что у decorative images (декоративных изображений) нет необходимости указывать текст. Какой атрибут alt должен быть указан?

Подсказка: Атрибут должен сигнализировать, что изображение не несёт смысловой нагрузки.

Ответ:

- **alt=""**
- alt="Декоративное изображение"

Шаг 04:

Миссия 4

 Сопоставьте атрибуты с их назначением для доступности.

Подсказка: Подумайте, какой атрибут для чего используется.

Ответ:

- **alt** — Описание изображения для экранных читалок

- **aria-label** — Текстовое описание элемента интерфейса
- **aria-hidden** — Скрытие элемента от вспомогательных технологий
- **title** — Всплывающая подсказка при наведении
- **role** — Определяет семантическую роль элемента

🔮 Шаг 05:

Миссия 5

📖 Сопоставьте сценарий использования с правильным атрибутом.

Подсказка: Подумайте, какой атрибут улучшает доступность в каждой ситуации.

Ответ:

- **Кнопка с иконкой без текста** — `aria-label`
- **Декоративное изображение** — `alt=""`
- **Интерактивный элемент, скрытый для всех** — `aria-hidden="true"`
- **Заголовок статьи** — `role="heading"`
- **Текст подсказки при наведении** — `title`

🔮 Шаг 06:

Миссия 6

Сергей проверяет форму. Какой атрибут `aria` поможет обозначить, что поле обязательно к заполнению?

Подсказка: Этот атрибут указывает состояние элемента для вспомогательных технологий.

Ответ:

- `aria-required="true"`
- `required="false"`

🔮 Шаг 07:

Миссия 7

Наталья создает меню навигации. Какой атрибут `aria` используется, чтобы указать раскрывающееся состояние?

Подсказка: Атрибут начинается с `"aria-"` и обозначает состояние.

Ответ:

- `aria-expanded`
- `aria-hidden`

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте атрибут `aria` с его состоянием или ролью.

Подсказка: Подумайте, что именно описывает атрибут.

Ответ:

- **`aria-checked`** — Состояние флажка или переключателя
- **`aria-selected`** — Выбранный элемент списка
- **`aria-disabled`** — Элемент недоступен для взаимодействия
- **`aria-hidden`** — Элемент скрыт для вспомогательных технологий
- **`aria-expanded`** — Раскрытое или свернутое состояние

🔗 Шаг 09:

Миссия 9

Дмитрий добавляет картинку кнопки. Как сделать так, чтобы экранные читалки не читали лишние декоративные элементы?

Подсказка: Используется пустой alt или aria-атрибут.

Ответ:

- **alt=""**
- alt="Кнопка"

🔗 Шаг 10:

Миссия 10

Елена хочет, чтобы пользователь понимал роль каждого блока навигации. Какой атрибут добавить?

Подсказка: Атрибут определяет семантическую роль элемента.

Ответ:

- **role="navigation"**
- alt="Навигация"

🔗 Шаг 11:

Миссия 11

📄 Сопоставьте элемент с правильным aria-атрибутом для улучшения доступности.

Подсказка: Подумайте о типичном использовании этих элементов.

Ответ:

- **input checkbox** — aria-checked
- **select option** — aria-selected
- **fieldset недоступен** — aria-disabled
- **декоративное изображение** — aria-hidden
- **кнопка раскрытия меню** — aria-expanded

🔗 Шаг 12:

Миссия 12

Максим проверяет страницу через экранный читалку. Какой alt-текст для логотипа компании будет правильным?

Подсказка: Он должен быть коротким и информативным.

Ответ:

- **alt="Логотип компании XYZ"**
- alt="Изображение"
- alt=""

🔗 Шаг 13:

Миссия 13

Расположите шаги добавления доступного изображения в правильном порядке:

1. Выбираем изображение
2. Добавляем атрибут alt с описанием

3. Если изображение декоративное, оставляем alt пустым

4. Проверяем работу через экранный читалку

Подсказка: Сначала выбираем изображение, затем делаем его доступным, и только потом проверяем.

Ответ:

- Выбираем изображение
- Добавляем атрибут alt с описанием
- Если изображение декоративное, оставляем alt пустым
- Проверяем работу через экранный читалку

Урок 2 - Навигация через tabindex

Шаг 01:

Миссия 1

Анна создает форму и хочет управлять порядком перехода по элементам при Tab. Какой атрибут для этого используют?

Подсказка: Этот атрибут определяет порядок навигации с клавиатуры.

Ответ:

- **tabindex**
- accesskey
- role

Шаг 02:

Миссия 2

Иван добавил кнопку с tabindex="-1". Что это означает?

Подсказка: Элемент не должен быть доступен при стандартной навигации клавиатурой.

Ответ:

- **Элемент исключен из последовательности Tab**
- Элемент выделяется первым при Tab

Шаг 03:

Миссия 3

Ольга хочет, чтобы первый input был доступен первым при переходе клавишей Tab. Какой tabindex нужно указать?

Подсказка: Обычно для стандартного порядка используется значение 0.

Ответ:

- **tabindex="1"**
- tabindex="-1"

Шаг 04:

Миссия 4

 Сопоставьте значения tabindex с их поведением.

Подсказка: Подумайте, как клавиша Tab будет вести себя для каждого значения.

Ответ:

- **tabindex="0"** — Элемент следует в стандартном порядке навигации
- **tabindex="-1"** — Элемент исключен из последовательности Tab
- **tabindex="1"** — Элемент будет первым при навигации Tab
- **tabindex="2"** — Следующий элемент после tabindex=1
- **tabindex="3"** — Следующий элемент после tabindex=2

Шаг 05:

Миссия 5

 Сопоставьте элементы формы с правильным поведением при Tab.

Подсказка: У каждого элемента свой tabindex или его отсутствие.

Ответ:

- **Поле ввода имени, tabindex=1** — Переходит первым
- **Кнопка отправки, tabindex=3** — Переходит последней
- **Ссылка на сайт, tabindex=2** — Переходит второй
- **Информация, tabindex=-1** — Не доступна при Tab
- **Доп. поле ввода, tabindex=0** — Следует в стандартном порядке

Шаг 06:

Миссия 6

Сергей хочет временно убрать кнопку из Tab-порядка, но оставить её видимой. Какой tabindex указать?

Подсказка: Элемент должен быть видимым, но не доступным через Tab.

Ответ:

- **tabindex="-1"**
- **tabindex="0"**

Шаг 07:

Миссия 7

Алина хочет, чтобы навигация через Tab шла по визуальному порядку элементов. Какой tabindex нужно использовать?

Подсказка: Значение 0 добавляет элемент в естественный порядок.

Ответ:

- **tabindex="0"**
- **tabindex="-1"**

Шаг 08:

Миссия 8

 Сопоставьте элементы интерфейса с их tabindex-поведением.

Подсказка: Подумайте, как пользователь будет перемещаться по элементам с клавиатуры.

Ответ:

- **Кнопка меню, tabindex=1** — Переходит первой
- **Поле поиска, tabindex=2** — Следует после меню
- **Декоративное изображение, tabindex=-1** — Не участвует в Tab-переходе

- **Основной контент, `tabindex=0`** — Следует в естественном порядке
- **Ссылка на контакт, `tabindex=3`** — Переходит после поля поиска

Шаг 09:

Миссия 9

Вадим тестирует страницу: все элементы с `tabindex>0`. Что нужно проверить в первую очередь?
Подсказка: Порядок чисел `tabindex` определяет последовательность навигации.

Ответ:

- **Порядок чисел соответствует логике интерфейса**
- Все элементы имеют одинаковое значение `tabindex`

Шаг 10:

Миссия 10

Наталья хочет, чтобы скрытый элемент был недоступен для Tab. Какой `tabindex` использовать?
Подсказка: Элемент не должен быть доступен клавиатурой.

Ответ:

- **`tabindex="-1"`**
- `tabindex="0"`

Шаг 11:

Миссия 11

 Сопоставьте элементы формы с их `tabindex`-значениями.

Подсказка: Подумайте, какой элемент должен идти первым, а какой исключён.

Ответ:

- **Поле email, `tabindex=1`** — Переходит первым
- **Поле пароль, `tabindex=2`** — Переходит вторым
- **Информационный текст, `tabindex=-1`** — Не участвует в Tab
- **Кнопка входа, `tabindex=3`** — Переходит последней
- **Ссылка на регистрацию, `tabindex=0`** — Следует в естественном порядке

Шаг 12:

Миссия 12

Расположите шаги корректной настройки `tabindex` для формы:

1. Определяем порядок логики формы
2. Назначаем `tabindex` для каждого элемента
3. Исключаем декоративные элементы `tabindex=-1`
4. Проверяем последовательность Tab вручную

Подсказка: Сначала планируем, затем назначаем, затем проверяем.

Ответ:

- Определяем порядок логики формы
- Назначаем `tabindex` для каждого элемента
- Исключаем декоративные элементы `tabindex=-1`
- Проверяем последовательность Tab вручную

Шаг 13:

Миссия 13

Расположите действия для тестирования навигации Tab:

1. Переходим по элементам клавишей Tab
2. Отмечаем элементы, которые не должны быть в Tab-порядке
3. Исправляем tabindex элементов, нарушающих порядок
4. Повторяем проверку, пока порядок не станет логичным

Подсказка: Тестирование идёт по циклу: проверка, исправление, повторная проверка.

Ответ:

- Переходим по элементам клавишей Tab
- Отмечаем элементы, которые не должны быть в Tab-порядке
- Исправляем tabindex элементов, нарушающих порядок
- Повторяем проверку, пока порядок не станет логичным

Шаг 14:

Миссия 14

Расположите этапы настройки доступной клавиатурной навигации:

1. Выделяем интерактивные элементы
2. Определяем естественный порядок
3. Назначаем tabindex и исключаем лишние
4. Тестируем Tab-навигацию с экранной читалкой

Подсказка: Сначала выбираем элементы, затем настраиваем порядок и тестируем.

Ответ:

- Выделяем интерактивные элементы
- Определяем естественный порядок
- Назначаем tabindex и исключаем лишние
- Тестируем Tab-навигацию с экранной читалкой

Урок 3 - Screen readers и проверка

Шаг 01:

Миссия 1

Алина проверяет сайт с помощью экранного читалки. Что такое screen reader?

Подсказка: Подумайте, какой инструмент помогает людям с нарушением зрения использовать сайт.

Ответ:

- **Программа, которая озвучивает содержимое страницы**
- Плагин для ускоренной загрузки страниц
- Инструмент для проверки SEO

Шаг 02:

Миссия 2

Иван тестирует формы. Какой атрибут помогает screen reader правильно озвучивать поля ввода?

Подсказка: Обычно он содержит текстовое описание элемента.

Ответ:

- **aria-label**
- tabindex
- role

Шаг 03:

Миссия 3

Ольга хочет, чтобы screen reader игнорировал декоративное изображение. Какой атрибут она должна использовать?

Подсказка: Элемент должен быть скрыт от озвучивания.

Ответ:

- **aria-hidden="true"**
- aria-label=""

Шаг 04:

Миссия 4

 Сопоставьте элементы страницы с их правильным поведением для screen reader.

Подсказка: Подумайте, как пользователь с экранной читалкой будет воспринимать каждый элемент.

Ответ:

- **Декоративное изображение** — Не озвучивается
- **Кнопка отправки формы** — Озвучивается с aria-label или текстом
- **Поле ввода имени** — Озвучивается с помощью label или aria-label
- **Ссылка на главную** — Озвучивается как интерактивный элемент
- **Скрытый текст для подсказки** — Не озвучивается, если aria-hidden=true

Шаг 05:

Миссия 5

 Сопоставьте атрибуты ARIA с их назначением.

Подсказка: Каждый атрибут служит конкретной цели для доступности.

Ответ:

- **aria-label** — Добавляет текстовое описание для элемента
- **aria-hidden** — Скрывает элемент от screen reader
- **role="button"** — Определяет элемент как кнопку
- **aria-live** — Сообщает об изменениях динамического контента
- **aria-describedby** — Связывает элемент с описанием

Шаг 06:

Миссия 6

Сергей проверяет страницу с NVDA. Он видит, что кнопка не озвучивается. Что проверить в первую очередь?

Подсказка: Возможно, элемент не имеет текста или aria-label.

Ответ:

- Проверить наличие **aria-label** или текста внутри кнопки
- Проверить CSS-цвет кнопки

 **Шаг 07:**

Миссия 7

Алина тестирует динамическое уведомление. Как сделать так, чтобы screen reader сообщил об изменении?

Подсказка: Используется специальный ARIA-атрибут для live-обновлений.

Ответ:

- **aria-live="polite"**
- **aria-hidden="true"**

 **Шаг 08:**

Миссия 8

 Сопоставьте сценарии с подходящим ARIA-атрибутом.

Подсказка: Подумайте, как screen reader узнает о каждой ситуации.

Ответ:

- **Обновление корзины на сайте** — **aria-live="polite"**
- **Декоративная иконка** — **aria-hidden="true"**
- **Кнопка закрытия модального окна** — **role="button"**
- **Текстовое поле поиска** — **aria-label** или **label**
- **Сообщение об ошибке** — **aria-describedby**

 **Шаг 09:**

Миссия 9

Иван проверяет список ссылок. Что screen reader озвучит в первую очередь?

Подсказка: Обратите внимание на порядок и видимость элементов.

Ответ:

- **Первый видимый интерактивный элемент с текстом**
- Скрытые элементы без **aria-hidden**
- Весь HTML-код страницы

 **Шаг 10:**

Миссия 10

Наталья хочет проверить, что экранная читалка корректно озвучивает модальное окно. Что важно проверить в первую очередь?

Подсказка: Элемент должен быть доступен и правильно идентифицирован.

Ответ:

- **Наличие **role="dialog"** и **focus** на модальном окне**
- CSS-анимация открытия

 **Шаг 11:**

Миссия 11

Расположите шаги проверки страницы с screen reader:

1. Запустить screen reader
2. Навигация по интерактивным элементам
3. Проверка озвучивания всех важных элементов
4. Исправление элементов, не озвучиваемых корректно

Подсказка: Сначала тестируем, затем исправляем ошибки.

Ответ:

- Запустить screen reader
- Навигация по интерактивным элементам
- Проверка озвучивания всех важных элементов
- Исправление элементов, не озвучиваемых корректно

Шаг 12:

Миссия 12

Расположите этапы настройки доступности для динамического контента:

1. Добавление aria-live для уведомлений
2. Проверка через screen reader
3. Тестирование на разных браузерах
4. Корректировка текста и атрибутов при необходимости

Подсказка: Сначала добавляем атрибут, потом проверяем и исправляем.

Ответ:

- Добавление aria-live для уведомлений
- Проверка через screen reader
- Тестирование на разных браузерах
- Корректировка текста и атрибутов при необходимости

Шаг 13:

Миссия 13

Расположите действия для комплексной проверки доступности сайта:

1. Проверка навигации с клавиатуры
2. Проверка через screen reader
3. Исправление элементов с неправильным ARIA или tabindex
4. Повторная проверка после исправлений

Подсказка: Цикл проверки и исправления помогает сделать сайт полностью доступным.

Ответ:

- Проверка навигации с клавиатуры
- Проверка через screen reader
- Исправление элементов с неправильным ARIA или tabindex
- Повторная проверка после исправлений

Шаг 14:

Миссия 14

Андрей тестирует форму с динамическим контентом. Какой ARIA-атрибут позволит screen reader уведомить пользователя об изменении?

Подсказка: Это атрибут для live-обновлений контента.

Ответ:

- **aria-live="assertive"**
- **aria-hidden="true"**

Шаг 15:

Миссия 15

 Сопоставьте сценарии тестирования с ожидаемым поведением screen reader.

Подсказка: Подумайте, как элементы должны озвучиваться для доступности.

Ответ:

- **Ошибка при заполнении формы** — Озвучивается через aria-describedby
- **Обновление корзины** — Озвучивается через aria-live
- **Декоративная иконка** — Не озвучивается
- **Поле ввода с label** — Озвучивается с текстом label
- **Кнопка отправки формы** — Озвучивается как интерактивный элемент

Урок 4 - Контраст и читаемость текста

Шаг 01:

Миссия 1

Аня заметила, что текст на сайте сливается с фоном. Какой принцип доступности нарушен?

Подсказка: Подумайте, что помогает людям с нарушением зрения различать текст и фон.

Ответ:

- **Контраст текста и фона**
- Размер шрифта
- Анимация кнопок

Шаг 02:

Миссия 2

Владислав делает новый дизайн кнопок. Какой минимальный контраст текста на кнопке рекомендуют для удобочитаемости?

Подсказка: Смотрите на коэффициент контраста между текстом и фоном.

Ответ:

- **4.5:1 для нормального текста**
- 1:1
- 2:1

Шаг 03:

Миссия 3

Мария заметила, что серый текст на светлом фоне трудно читается. Что можно сделать?

Подсказка: Подумайте о контрасте и насыщенности цвета.

Ответ:

- **Увеличить контраст между текстом и фоном**
- Сделать фон прозрачным
- Добавить анимацию текста

Шаг 04:

Миссия 4

 Сопоставьте элементы интерфейса с действием для повышения читаемости.

Подсказка: Подумайте, что улучшает видимость текста на разных фонах.

Ответ:

- **Кнопка с бледным текстом** — Увеличить насыщенность цвета текста
- **Фон с градиентом под текстом** — Добавить тень или сплошной фон под текст
- **Мелкий текст** — Увеличить размер шрифта
- **Текст на изображении** — Добавить контрастный полупрозрачный слой
- **Список ссылок** — Использовать подчеркивание и контрастный цвет

Шаг 05:

Миссия 5

Сергей проверяет дизайн сайта. Какой инструмент поможет проверить контраст текста?

Подсказка: Подумайте о специализированных онлайн-сервисах и плагинах.

Ответ:

- **Контрастные анализаторы, например WebAIM Contrast Checker**
- Инспектор CSS без проверки контраста
- Браузер без расширений

Шаг 06:

Миссия 6

Лена работает над мобильной версией сайта. Что особенно важно для читаемости текста на маленьких экранах?

Подсказка: Подумайте о размере и контрасте текста.

Ответ:

- **Достаточный размер шрифта и высокий контраст с фоном**
- Использовать мелкий шрифт и светлый текст
- Текст одного цвета с фоном

Шаг 07:

Миссия 7

 Сопоставьте шрифты с их влиянием на читаемость текста.

Подсказка: Подумайте, какие шрифты легко читать на экране, а какие — нет.

Ответ:

- **Шрифт с засечками (serif) маленький** — Может ухудшать читаемость на экране
- **Шрифт без засечек (sans-serif)** — Легко читается на экране
- **Очень декоративный шрифт** — Трудно читается
- **Полужирный текст** — Повышает читаемость

- **Курсивный текст** — Может снижать читаемость при мелком размере

Шаг 08:

Миссия 8

Павел добавляет подсветку ссылок на сайте. Как это поможет доступности?

Подсказка: Подумайте, как пользователь видит интерактивные элементы.

Ответ:

- **Повышает видимость ссылок и удобство навигации**
- Меняет цвет текста везде на сайте
- Скрывает текст от пользователей

Шаг 09:

Миссия 9

Расположите действия по улучшению читаемости текста на сайте в правильном порядке:

1. Проверка контраста текста и фона
2. Выбор удобного шрифта и размера
3. Добавление подсветки интерактивных элементов
4. Проверка читаемости на разных устройствах

Подсказка: Сначала анализируем, затем применяем и проверяем изменения.

Ответ:

- Проверка контраста текста и фона
- Выбор удобного шрифта и размера
- Добавление подсветки интерактивных элементов
- Проверка читаемости на разных устройствах

Шаг 10:

Миссия 10

Расположите шаги для проверки доступности текста на изображении:

1. Определить, является ли текст важной информацией
2. Добавить контрастный фон или слой под текстом
3. Проверить читаемость с разными цветами и размерами
4. Использовать альтернативный текст для screen reader

Подсказка: Сначала анализируем, потом улучшаем визуально, потом проверяем.

Ответ:

- Определить, является ли текст важной информацией
- Добавить контрастный фон или слой под текстом
- Проверить читаемость с разными цветами и размерами
- Использовать альтернативный текст для screen reader

Шаг 11:

Миссия 11

Расположите шаги по оптимизации читаемости форм:

1. Выбор контрастного текста и фона

2. Добавление читаемых шрифтов
3. Проверка читаемости для разных размеров экранов
4. Тестирование с пользователями

Подсказка: Оптимизация форм требует анализа, внедрения и тестирования.

Ответ:

- Выбор контрастного текста и фона
- Добавление читаемых шрифтов
- Проверка читаемости для разных размеров экранов
- Тестирование с пользователями

Модуль 8 - ⚡ Оптимизация

Урок 1 - Lazy loading картинок

Шаг 01:

Миссия 1

Алексей заметил, что сайт долго грузится из-за большого числа изображений. Какое решение поможет ускорить загрузку?

Подсказка: Подумайте о том, чтобы загружать изображения только при необходимости.

Ответ:

- **Использовать lazy loading для картинок**
- Сразу загружать все изображения
- Увеличить размер изображений

Шаг 02:

Миссия 2

Марина добавляет атрибут loading к картинкам. Какое значение следует использовать для ленивой загрузки?

Подсказка: Подумайте, какой параметр откладывает загрузку изображения до прокрутки.

Ответ:

- **lazy**
- eager
- auto

Шаг 03:

Миссия 3

Игорь замечает, что пользователи на мобильных устройствах жалуются на медленную загрузку галереи. Что поможет?

Подсказка: Подумайте, как уменьшить объем данных, загружаемых сразу.

Ответ:

- **Применить lazy loading к картинкам галереи**
- Увеличить качество всех изображений
- Удалить галерею

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте атрибуты `img` с их ролью для оптимизации загрузки.

Подсказка: Подумайте, какие атрибуты помогают откладывать загрузку изображений.

Ответ:

- **loading** — Определяет стратегию загрузки изображения
- **src** — Указывает путь к изображению
- **alt** — Обеспечивает доступность и альтернативный текст
- **width/height** — Позволяют зарезервировать место до загрузки изображения
- **decoding** — Позволяет отложить декодирование изображения

🔧 Шаг 05:

Миссия 5

📖 Сопоставьте метод оптимизации с эффектом на производительность страницы.

Подсказка: Обратите внимание, что `lazy loading` уменьшает нагрузку на сеть.

Ответ:

- **Lazy loading изображений** — Снижает начальное время загрузки страницы
- **Сжатие изображений** — Уменьшает вес файлов
- **Использование CDN** — Повышает скорость доставки контента
- **Кеширование изображений** — Снижает повторные загрузки
- **Размер изображения по экрану** — Снижает трафик

🔧 Шаг 06:

Миссия 6

Борис применяет `lazy loading`, но изображения на первом экране не видны. Почему?

Подсказка: Подумайте, какие изображения должны загружаться сразу.

Ответ:

- **Первые важные изображения должны загружаться сразу (eager)**
- `lazy loading` нужно применить ко всем изображениям
- Нужно убрать атрибут `src`

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте библиотеку или метод `lazy loading` с его особенностью.

Подсказка: Обратите внимание на нативный атрибут и JS-решения.

Ответ:

- **Нативный `loading="lazy"`** — Простая реализация без библиотек
- **Intersection Observer API** — Позволяет загружать контент при появлении в viewport
- **lozad.js** — Легкая JS-библиотека для ленивой загрузки
- **lazysizes** — Поддержка responsive и `lazy loading`
- **Полифиллы для старых браузеров** — Обеспечивают совместимость с отсутствием нативного `lazy loading`

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте потенциальную проблему с решением при lazy loading.

Подсказка: Подумайте, что делать, если изображения не видны для поисковых роботов.

Ответ:

- **SEO-проблемы** — Использовать с изображениями
- **Мигающие изображения при скролле** — Добавить плавное появление (fade-in)
- **Проблемы с placeholder** — Убедиться, что зарезервировано место с width/height
- **Не загружаются первые изображения** — Применять eager для выше fold

🔗 Шаг 09:

Миссия 9

Ирина хочет использовать lazy loading для видео и изображений. Что нужно учитывать?

Подсказка: Подумайте, отличается ли поведение видео и img при отложенной загрузке.

Ответ:

- **Видео может требовать кастомный JS, а img поддерживает нативный loading="lazy"**
- Видео всегда поддерживает loading="lazy"
- Нельзя использовать lazy loading на видео

🔗 Шаг 10:

Миссия 10

Расположите шаги внедрения lazy loading в правильном порядке:

1. Анализируем, какие изображения можно загружать позже
2. Добавляем атрибут loading="lazy" или JS-скрипт
3. Тестируем работу на разных устройствах и браузерах
4. Оптимизируем размеры и форматы изображений

Подсказка: Сначала определяем, потом внедряем, проверяем и оптимизируем.

Ответ:

- Анализируем, какие изображения можно загружать позже
- Добавляем атрибут loading="lazy" или JS-скрипт
- Тестируем работу на разных устройствах и браузерах
- Оптимизируем размеры и форматы изображений

🔗 Шаг 11:

Миссия 11

Расположите шаги по обеспечению доступности при lazy loading:

1. Добавляем alt-текст для всех изображений
2. Используем для SEO и старых браузеров
3. Указываем размеры width и height для placeholder
4. Проверяем скринридерами и инструментами доступности

Подсказка: Сначала контент доступен, потом SEO, потом визуальные размеры, потом проверка.

Ответ:

- Добавляем alt-текст для всех изображений
- Используем для SEO и старых браузеров
- Указываем размеры width и height для placeholder
- Проверяем скринридерами и инструментами доступности

 Шаг 12:

Миссия 12

Расположите шаги по отладке lazy loading галереи:

1. Проверка отображения изображений при скролле
2. Оптимизация форматов изображений и сжатие
3. Добавление fade-in анимации для плавного появления
4. Тестирование на мобильных устройствах и слабом интернете

Подсказка: Сначала базовая проверка, затем оптимизация, анимация и тестирование.

Ответ:

- Проверка отображения изображений при скролле
- Оптимизация форматов изображений и сжатие
- Добавление fade-in анимации для плавного появления
- Тестирование на мобильных устройствах и слабом интернете

 Шаг 13:

Миссия 13

Виталий хочет измерить эффект lazy loading на скорости загрузки. Что нужно сделать?

Подсказка: Используйте инструменты измерения времени загрузки и производительности.

Ответ:

- **Использовать Lighthouse или DevTools Network для проверки времени загрузки**
- Проверить только визуально
- Удалить все изображения и сравнить

 Урок 2 - Использование WebP и AVIF

 Шаг 01:

Миссия 1

Алексей заметил, что изображения на сайте занимают много места и медленно загружаются. Какой формат стоит использовать для оптимизации?

Подсказка: Подумайте о современных форматах изображений с меньшим размером.

Ответ:

- **WebP или AVIF**
- JPEG или PNG без изменений
- BMP

 Шаг 02:

Миссия 2

Марина хочет улучшить скорость загрузки галереи. Какой подход будет правильным?

Подсказка: Подумайте о форматах изображений, которые уменьшают вес файлов без потери качества.

Ответ:

- **Конвертировать изображения в WebP или AVIF**
- Использовать только PNG
- Оставить JPEG с высоким разрешением

Шаг 03:

Миссия 3

Игорь хочет использовать AVIF для изображений, но некоторые браузеры его не поддерживают. Что сделать?

Подсказка: Подумайте о механизме резервного варианта для старых браузеров.

Ответ:

- **Использовать с fallback на WebP или JPEG**
- Оставить только AVIF
- Конвертировать AVIF в BMP

Шаг 04:

Миссия 4

 Сопоставьте формат изображения с его преимуществом.

Подсказка: Подумайте, какой формат более современный и эффективный.

Ответ:

- **JPEG** — Широко поддерживается, но размер файлов больше
- **PNG** — Хорош для прозрачности, но тяжелее
- **WebP** — Меньший размер при сохранении качества, поддержка большинства браузеров
- **AVIF** — Наименьший размер и высокая эффективность, современный формат

Шаг 05:

Миссия 5

 Сопоставьте формат изображения с его особенностью сжатия.

Подсказка: Подумайте, какой формат использует современные алгоритмы сжатия.

Ответ:

- **JPEG** — С потерями качества, но эффективное сжатие
- **PNG** — Без потерь, но файлы тяжелее
- **WebP** — Сжатие с потерями и без потерь, меньший размер
- **AVIF** — Высокоэффективное сжатие с потерями и без потерь

Шаг 06:

Миссия 6

Анна хочет, чтобы изображения сайта занимали меньше трафика для мобильных пользователей. Что лучше применить?

Подсказка: Подумайте, какой формат снижает вес файлов без заметной потери качества.

Ответ:

- **Конвертировать изображения в WebP или AVIF**

- Использовать только JPEG высокого качества
- Использовать BMP

🔮 Шаг 07:

Миссия 7

Игорь внедряет AVIF на сайт, но часть пользователей не видит изображения. Как исправить?
Подсказка: Подумайте о резервном формате для несовместимых браузеров.

Ответ:

- **Использовать с fallback на WebP или JPEG**
- Оставить только AVIF
- Удалить изображения

🔮 Шаг 08:

Миссия 8

📄 Сопоставьте подход к внедрению современных форматов с его преимуществом.
Подсказка: Подумайте о том, как снизить нагрузку на сеть и увеличить скорость.

Ответ:

- **Использовать WebP** — Снижение веса файлов, быстрый рендер
- **Использовать AVIF** — Максимальная экономия трафика, высокая эффективность
- **Использовать fallback JPEG** — Совместимость со всеми браузерами

🔮 Шаг 09:

Миссия 9

📄 Сопоставьте оптимизацию изображений с ее эффектом на производительность.
Подсказка: Подумайте, что уменьшение размера файлов делает с загрузкой страницы.

Ответ:

- **Сжатие WebP/AVIF** — Уменьшение времени загрузки
- **Использование современных форматов** — Меньше трафика для пользователей
- **Fallback форматы** — Совместимость с разными браузерами
- **Оптимизация размеров** — Быстрое отображение и меньше скrolла

🔮 Шаг 10:

Миссия 10

Виталий хочет, чтобы изображения на сайте были одновременно быстрыми и качественными.
Какой подход выбрать?
Подсказка: Подумайте о сочетании формата и размера файлов.

Ответ:

- **WebP/AVIF с оптимизацией размеров**
- Оставить PNG высокого разрешения
- Использовать BMP

🔮 Шаг 11:

Миссия 11

Расположите шаги внедрения WebP/AVIF в правильном порядке:

1. Анализ существующих изображений и их форматов
2. Конвертация в WebP или AVIF с оптимизацией размеров
3. Добавление с fallback для несовместимых браузеров
4. Тестирование отображения и производительности

Подсказка: Сначала анализируем, потом конвертируем, обеспечиваем fallback и проверяем.

Ответ:

- Анализ существующих изображений и их форматов
- Конвертация в WebP или AVIF с оптимизацией размеров
- Добавление с fallback для несовместимых браузеров
- Тестирование отображения и производительности

Урок 3 - Минификация HTML-кода

Шаг 01:

Миссия 1

Алексей заметил, что HTML-код сайта слишком большой и страницы загружаются медленно. Что поможет уменьшить размер кода?

Подсказка: Подумайте о способе убрать лишние пробелы и комментарии.

Ответ:

- **Минификация HTML**
- Добавление комментариев
- Использование таблиц для верстки
- Увеличение отступов

Шаг 02:

Миссия 2

Марина хочет ускорить загрузку сайта и уменьшить трафик пользователей. Какой инструмент лучше применить?

Подсказка: Подумайте о способе автоматической оптимизации кода без изменения его функционала.

Ответ:

- **Минификатор HTML-кода**
- Редактор текста
- Браузерная консоль
- CSS-фреймворк

Шаг 03:

Миссия 3

Игорь решил уменьшить размер HTML вручную. Почему это не лучший подход?

Подсказка: Подумайте о времени и вероятности ошибок.

Ответ:

- **Процесс займет много времени и возможны ошибки**
- HTML автоматически станет быстрее
- Браузеры не смогут прочитать код

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте термины минификации с их объяснением.

Подсказка: Подумайте, что делает код меньше без потери функционала.

Ответ:

- **Минификация** — Удаление пробелов, переносов строк и комментариев
- **Обфускация** — Сделать код трудночитаемым для защиты
- **Компрессия** — Сжатие кода на сервере перед отправкой пользователю
- **Оптимизация** — Улучшение структуры кода для скорости и читаемости

🔧 Шаг 05:

Миссия 5

📖 Сопоставьте этапы минификации с их назначением.

Подсказка: Сначала упрощаем код, потом проверяем его работоспособность.

Ответ:

- **Удаление комментариев** — Снижение размера HTML
- **Удаление лишних пробелов** — Уменьшение объема файла
- **Проверка валидности** — Убедиться, что код работает правильно
- **Оптимизация inline-кода** — Ускорение рендеринга

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте типы файлов с их минификацией.

Подсказка: HTML можно минифицировать отдельно от CSS и JS.

Ответ:

- **HTML** — Удаление пробелов и комментариев, сохранение структуры
- **CSS** — Удаление пробелов, сокращение свойств
- **JS** — Удаление пробелов, сокращение имен переменных
- **JSON** — Удаление пробелов и переносов строк

🔧 Шаг 07:

Миссия 7

Анна хочет уменьшить размер HTML страницы без потери функциональности. Что лучше использовать?

Подсказка: Подумайте о специальных инструментах для этого процесса.

Ответ:

- **Автоматический минификатор HTML**
- Ручное удаление тегов
- Увеличение отступов

🔧 Шаг 08:

Миссия 8

Игорь заметил, что минифицированный HTML быстрее загружается. Почему?

Подсказка: Подумайте, как размер файла влияет на скорость загрузки.

Ответ:

- **Меньший размер файла загружается быстрее**
- Браузер ускоряет обработку CSS
- Минификация улучшает дизайн страницы

 **Шаг 09:**

Миссия 9

 Сопоставьте преимущества минификации с эффектом на сайт.

Подсказка: Подумайте о времени загрузки и размере страницы.

Ответ:

- **Меньший размер HTML** — Быстрая загрузка страниц
- **Удаление пробелов и комментариев** — Снижение трафика пользователей
- **Оптимизация inline-кода** — Ускорение рендеринга браузером
- **Автоматизация процесса** — Меньше ошибок и экономия времени

 **Шаг 10:**

Миссия 10

 Сопоставьте подходы к минификации с их эффектом на разработку.

Подсказка: Подумайте, как автоматизация помогает разработчику.

Ответ:

- **Ручная минификация** — Занимает много времени и подвержена ошибкам
- **Автоматическая минификация** — Быстро и безопасно, без ошибок
- **Обфускация** — Усложняет чтение кода для защиты
- **Комбинированный подход** — Минификация с тестированием работоспособности

Урок 4 - Тестирование скорости загрузки

 **Шаг 01:**

Миссия 1

Алекс решил проверить скорость загрузки своего сайта. Какой инструмент ему подойдет?

Подсказка: Подумайте о сервисах, которые показывают время загрузки страницы.

Ответ:

- **Google PageSpeed Insights**
- Блокнот
- Photoshop
- Excel

 **Шаг 02:**

Миссия 2

Марина хочет узнать, какие элементы замедляют загрузку страницы. Какой инструмент использовать?

Подсказка: Ищите инструменты, которые дают детальный отчет по ресурсам.

Ответ:

- **Lighthouse**

- Word
- Paint
- Notepad++

Шаг 03:

Миссия 3

Игорь хочет оценить скорость загрузки сайта на мобильных устройствах. Что нужно учитывать?
Подсказка: Подумайте о реальном пользовательском опыте на смартфонах.

Ответ:

- **Скорость загрузки и оптимизацию под мобильные сети**
- Только размер изображений
- Только шрифты

Шаг 04:

Миссия 4

 Сопоставьте понятия тестирования скорости с их объяснением.

Подсказка: Обратите внимание, что один термин отвечает за измерение, другой — за оптимизацию.

Ответ:

- **PageSpeed Insights** — Инструмент для анализа скорости и рекомендаций по оптимизации
- **Lighthouse** — Глубокий аудит сайта с отчетом по производительности
- **Время первого байта (TTFB)** — Метрика, показывающая задержку ответа сервера
- **Largest Contentful Paint (LCP)** — Показатель времени загрузки основного контента

Шаг 05:

Миссия 5

 Сопоставьте метрики скорости с их значением для пользователя.

Подсказка: Думайте, какая метрика влияет на восприятие скорости сайта.

Ответ:

- **First Contentful Paint (FCP)** — Время появления первого элемента контента на экране
- **Time to Interactive (TTI)** — Время, когда страница становится полностью интерактивной
- **Cumulative Layout Shift (CLS)** — Сдвиг элементов во время загрузки страницы
- **Total Blocking Time (TBT)** — Суммарное время, когда пользователь не может взаимодействовать с сайтом

Шаг 06:

Миссия 6

 Сопоставьте инструменты тестирования скорости с их особенностями.

Подсказка: Некоторые инструменты дают визуализацию, другие — аналитические отчеты.

Ответ:

- **WebPageTest** — Подробный отчет с визуализацией загрузки всех ресурсов
- **GTmetrix** — Сводка по скорости и рекомендации по оптимизации
- **Pingdom Tools** — Быстрая оценка времени загрузки страницы
- **Chrome DevTools** — Анализ сети и производительности прямо в браузере

Шаг 07:

Миссия 7

Анна хочет проверить, как оптимизация изображений влияет на скорость сайта. Что нужно сделать?

Подсказка: Сравните показатели до и после изменений.

Ответ:

- **Протестировать сайт до и после оптимизации изображений**
- Изменить шрифты
- Добавить анимацию

Шаг 08:

Миссия 8

Иван заметил, что мобильная версия сайта загружается медленно. Что стоит проверить в первую очередь?

Подсказка: Думайте о сетевых ограничениях и размерах ресурсов.

Ответ:

- **Размер изображений и скриптов для мобильных сетей**
- Цвет фона
- Шрифты заголовков

Шаг 09:

Миссия 9

 Сопоставьте оптимизационные действия с их влиянием на показатели скорости.

Подсказка: Некоторые действия уменьшают размер, другие ускоряют рендеринг.

Ответ:

- **Минификация CSS и JS** — Снижение общего размера страницы
- **Lazy loading изображений** — Уменьшение времени загрузки видимой области
- **Оптимизация изображений** — Снижение трафика и ускорение загрузки
- **Кэширование ресурсов** — Повторная загрузка страниц быстрее

Шаг 10:

Миссия 10

Расположите шаги тестирования скорости загрузки в правильном порядке:

1. Выбор инструмента тестирования
2. Сбор данных о загрузке страницы
3. Анализ результатов
4. Внесение оптимизаций
5. Повторное тестирование для проверки улучшений

Подсказка: Сначала измеряем, потом исправляем, потом проверяем снова.

Ответ:

- Выбор инструмента тестирования
- Сбор данных о загрузке страницы
- Анализ результатов

- Внесение оптимизаций
- Повторное тестирование для проверки улучшений

Модуль 9 - 🏛 HTML5+ фичи

Урок 1 - Элемент `dialog` для модалок

Шаг 01:

Миссия 1

Алекс создает модальное окно для подтверждения удаления записи. Какой элемент HTML ему стоит использовать?

Подсказка: Подумайте о встроенном теге для диалогов.

Ответ:

- **`dialog`**
- `section`
- `article`
- `div`

Шаг 02:

Миссия 2

Марина хочет открыть модалку по кнопке. Какой метод JavaScript ей нужен?

Подсказка: Элемент `dialog` имеет встроенный метод для показа.

Ответ:

- **`showModal()`**
- `alert()`
- `prompt()`

Шаг 03:

Миссия 3

Игорь хочет, чтобы модалка была скрыта при загрузке страницы. Какой атрибут нужно использовать?

Подсказка: Элемент `dialog` может быть скрыт по умолчанию.

Ответ:

- **`hidden`**
- `visible`
- `open`

Шаг 04:

Миссия 4

 Сопоставьте свойства и методы `dialog` с их описанием.

Подсказка: Обратите внимание, какие действия управляют показом и закрытием модалки.

Ответ:

- **`show()`** — Открывает диалоговое окно без модального режима
- **`showModal()`** — Открывает диалоговое окно модально, блокируя фон
- **`close()`** — Закрывает диалоговое окно

- **орен атрибут** — Показывает диалог при загрузке страницы

🔗 Шаг 05:

Миссия 5

📖 Сопоставьте элементы интерфейса модальки с их назначением.

Подсказка: Подумайте, какие кнопки нужны пользователю для взаимодействия с модалькой.

Ответ:

- **Кнопка «Закрыть»** — Закрывает модальное окно
- **Кнопка «Подтвердить»** — Подтверждает действие пользователя
- **Кнопка «Отмена»** — Отменяет действие и закрывает модальку
- **Фон модальки** — Блокирует взаимодействие с контентом страницы

🔗 Шаг 06:

Миссия 6

Анна хочет, чтобы модалька закрывалась при клике вне её. Как это можно реализовать?

Подсказка: Элемент `dialog` сам по себе не закрывается при клике вне, нужен JS.

Ответ:

- **Добавить обработчик клика на фон, вызывающий `close()`**
- Использовать атрибут `hidden`
- Использовать `show()`

🔗 Шаг 07:

Миссия 7

Борис хочет узнать, поддерживают ли старые браузеры тег `dialog`. Что ему делать?

Подсказка: Некоторые функции HTML5 не поддерживаются в старых версиях.

Ответ:

- **Использовать полифилл или проверку поддержки через JS**
- Просто использовать `dialog`, ничего не делая
- Заменить `dialog` на `div`

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте события модальки с их типичной обработкой.

Подсказка: Подумайте, какие события нужны для UI и UX.

Ответ:

- **`cancel`** — Вызывается при нажатии `Esc`, закрывает диалог
- **`close`** — Происходит после закрытия диалогового окна
- **клик на фон** — Можно добавить обработчик для закрытия модальки
- **`submit` формы внутри `dialog`** — Можно обработать подтверждение действия

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте атрибуты и их роль для диалогового окна.

Подсказка: Некоторые атрибуты управляют отображением, другие — поведением.

Ответ:

- **open** — Отображает диалоговое окно при загрузке
- **method** — Определяет способ отправки формы внутри dialog
- **novalidate** — Отключает валидацию формы внутри диалога
- **form** — Связывает диалог с определенной формой

🔗 **Шаг 10:**

Миссия 10

Светлана хочет, чтобы модалка была доступна для клавиатуры. Что важно сделать?

Подсказка: Подумайте про фокус и tab-index.

Ответ:

- **Установить фокус на первый элемент внутри dialog при открытии**
- Скрыть все кнопки
- Не использовать JavaScript

🔗 **Шаг 11:**

Миссия 11

Расположите шаги создания модалки с использованием dialog в правильном порядке:

1. Добавить элемент dialog в HTML
2. Создать кнопки для открытия и закрытия
3. Добавить обработчики JS (showModal, close)
4. Настроить стили и позиционирование
5. Протестировать модалку на разных устройствах

Подсказка: Сначала создаем структуру, потом добавляем логику и стили, затем тестируем.

Ответ:

- Добавить элемент dialog в HTML
- Создать кнопки для открытия и закрытия
- Добавить обработчики JS (showModal, close)
- Настроить стили и позиционирование
- Протестировать модалку на разных устройствах

📖 Урок 2 - Template и content для шаблонов

🔗 **Шаг 01:**

Миссия 1

Алекс создает HTML-шаблон для карточки товара, которая будет многократно клонироваться. Какой элемент он должен использовать?

Подсказка: Подумайте о теге для хранения разметки без отображения на странице.

Ответ:

- **template**
- section
- div

🔗 Шаг 02:

Миссия 2

Марина хочет поместить внутрь шаблона текст и изображения, которые будут вставлены на страницу позже. Какой элемент ей нужен?

Подсказка: Рассмотрите элемент, который позволяет хранить разметку до клонирования.

Ответ:

- **template**
- article
- div

🔗 Шаг 03:

Миссия 3

Игорь создает шаблон для всплывающих уведомлений. Какой тег нужен для хранения невидимого контента на странице?

Подсказка: Элемент не отображается до клонирования.

Ответ:

- **template**
- span
- section

🔗 Шаг 04:

Миссия 4

📄 Сопоставьте элементы шаблона с их назначением.

Подсказка: Подумайте, какие элементы управляют хранением разметки и отображением.

Ответ:

- **template** — Хранит разметку, которая не отображается до клонирования
- **content** — Предоставляет доступ к внутреннему содержимому шаблона
- **slot** — Используется для вставки динамического контента в шаблон
- **div** — Обычный контейнер, который сразу отображается

🔗 Шаг 05:

Миссия 5

📄 Сопоставьте методы JavaScript для работы с шаблонами с их действиями.

Подсказка: Подумайте, какие методы клонируют разметку и вставляют её на страницу.

Ответ:

- **cloneNode(true)** — Клонирует весь шаблон с содержимым
- **appendChild** — Вставляет клонированный узел в DOM
- **querySelector** — Находит шаблон по селектору
- **innerHTML** — Позволяет вставить HTML-код внутрь элемента

🔗 Шаг 06:

Миссия 6

Анна хочет использовать один шаблон для нескольких карточек продуктов. Как ей это сделать?

Подсказка: Элемент `template` позволяет многократное клонирование.

Ответ:

- **Клонировать template через cloneNode и вставлять в DOM**
- Копировать innerHTML вручную
- Использовать несколько div вместо template

🔧 **Шаг 07:**

Миссия 7

Борис хочет создать динамическую галерею изображений. Что следует использовать?

Подсказка: Шаблон позволяет подставлять разметку несколько раз.

Ответ:

- **template с последующим клонированием через JS**
- div с display:none
- article с fixed-контентом

🔧 **Шаг 08:**

Миссия 8

📖 Сопоставьте свойства template с их особенностями.

Подсказка: Внутреннее содержимое template хранится в специальном свойстве.

Ответ:

- **content** — Содержит DocumentFragment с разметкой шаблона
- **innerHTML** — Позволяет вставить HTML-код внутрь элемента
- **outerHTML** — Возвращает HTML вместе с самим элементом
- **cloneNode(true)** — Клонирует все содержимое template

🔧 **Шаг 09:**

Миссия 9

📖 Сопоставьте типичные ошибки при работе с template с последствиями.

Подсказка: Ошибки чаще связаны с неправильным клонированием или вставкой.

Ответ:

- **Не использовать cloneNode** — Все карточки будут ссылаться на один элемент, что ломает контент
- **Прямое вставление template без content** — Ничего не отобразится на странице
- **Использовать innerHTML вместо content** — Потеряются события и вложенные узлы

🔧 **Шаг 10:**

Миссия 10

Светлана хочет добавить динамический текст внутрь шаблона перед вставкой. Как это сделать?

Подсказка: Изменять содержимое можно на уровне клонированного узла.

Ответ:

- **Клонировать template и менять текст через querySelector перед вставкой**
- Изменять сам элемент template на странице
- Использовать alert для текста

🔧 Шаг 11:

Миссия 11

Антон хочет скрыть шаблон в DOM, чтобы он не отображался на странице. Какой тег подойдет?

Подсказка: Подумайте о невидимом контейнере для разметки.

Ответ:

- **template**
- section
- div

🔧 Шаг 12:

Миссия 12

📄 Сопоставьте этапы работы с шаблонами с действиями.

Подсказка: Сначала нужно найти, потом клонировать, затем вставить.

Ответ:

- **Найти template** — Использовать `querySelector` или `getElementById`
- **Клонировать template** — Вызвать `cloneNode(true)` для content
- **Изменить содержимое** — Добавить динамический текст или изображения
- **Вставить в DOM** — Использовать `appendChild` или `insertBefore`

📖 Урок 3 - Responsive images srcset и sizes

🔧 Шаг 01:

Миссия 1

Алиса создает адаптивные изображения для сайта. Какой атрибут позволяет браузеру выбирать подходящий размер изображения?

Подсказка: Один из атрибутов тега `img` отвечает за список вариантов.

Ответ:

- **srcset**
- sizes
- alt

🔧 Шаг 02:

Миссия 2

Борис хочет указать браузеру, как выбирать изображение в зависимости от ширины экрана. Какой атрибут ему нужен?

Подсказка: Этот атрибут используется вместе с `srcset`.

Ответ:

- **sizes**
- src
- alt

🔧 Шаг 03:

Миссия 3

Катя хочет сделать адаптивные изображения для карточек товаров. Что нужно добавить в `img`?

Подсказка: Один атрибут указывает размеры файла, другой — список источников.

Ответ:

- **srcset и sizes**
- alt и title
- width и height

💡 Шаг 04:

Миссия 4

📄 Сопоставьте атрибуты img с их функцией.

Подсказка: Подумайте, какой атрибут указывает список изображений, а какой — правила выбора размера.

Ответ:

- **srcset** — Список доступных изображений с указанием их ширины
- **sizes** — Правила выбора изображения в зависимости от размера viewport
- **alt** — Альтернативный текст для изображения
- **src** — Основной источник изображения

💡 Шаг 05:

Миссия 5

📄 Сопоставьте форматы srcset с их назначением.

Подсказка: В srcset можно указывать размеры в px и кратные масштабы.

Ответ:

- **1x, 2x** — Изображение для дисплеев с разной плотностью пикселей
- **300w, 600w** — Ширина изображения в пикселях для выбора по sizes
- **webp** — Современный формат сжатия для web
- **jpeg** — Стандартный формат для изображений

💡 Шаг 06:

Миссия 6

Иван хочет, чтобы на мобильных устройствах загружалось меньше данных. Что нужно сделать?

Подсказка: Браузер выбирает изображение на основе viewport.

Ответ:

- **Использовать srcset с разными ширинами и sizes**
- Добавить width и height одинаковые для всех
- Использовать только src

💡 Шаг 07:

Миссия 7

Светлана создает галерею изображений для десктопа и мобильных. Как правильно задать изображения?

Подсказка: Нужно задать несколько вариантов для разных размеров.

Ответ:

- **Указать srcset с ширинами и sizes с правилами**
- Добавить несколько img без srcset

- Использовать picture без source

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте единицы измерения в srcset с их назначением.

Подсказка: В srcset можно использовать ширину изображения и коэффициент масштаба.

Ответ:

- **w** — Ширина изображения в пикселях
- **x** — Плотность пикселей (коэффициент масштабирования)

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте ситуации с тем, какой srcset использовать.

Подсказка: Мобильное устройство требует меньший размер, а Retina экран — больший.

Ответ:

- **Обычный дисплей, малый экран** — Использовать меньшую ширину (например 300w)
- **Retina дисплей, большой экран** — Использовать большую ширину (например 1200w)

🔗 Шаг 10:

Миссия 10

Дмитрий хочет, чтобы изображения подстраивались под ширину родительского блока. Какой атрибут ему нужен?

Подсказка: Атрибут sizes определяет правила для ширины контейнера.

Ответ:

- **sizes**
- src
- alt

🔗 Шаг 11:

Миссия 11

Наталья хочет использовать несколько форматов изображений (webp и jpeg) для адаптивной галереи. Как это реализовать?

Подсказка: Рассмотрите элемент picture с source и media.

Ответ:

- **Использовать picture с несколькими source и media**
- Использовать только img с srcset
- Использовать div с background-image

📖 Урок 4 - Элемент picture и разные форматы

🔗 Шаг 01:

Миссия 1

Алиса хочет, чтобы на сайте отображались разные форматы изображений для современных и старых браузеров. Какой элемент HTML ей нужен?

Подсказка: Рассмотрите контейнер, который позволяет задавать несколько источников изображений.

Ответ:

- **picture**
- `img`
- `div`

🔮 Шаг 02:

Миссия 2

Борис хочет добавить изображение в формате WebP с fallback на JPEG для старых браузеров. Как правильно использовать элемент `picture`?

Подсказка: Один тег внутри `picture` задает источник изображения, другой — резервный `img`.

Ответ:

- **Использовать `source type="image/webp"` и `img` для JPEG**
- Использовать только `img` с WebP
- Использовать `div` с `background-image`

🔮 Шаг 03:

Миссия 3

Катя хочет, чтобы браузер сам выбирал подходящий размер изображения в зависимости от ширины экрана. Какие атрибуты нужно использовать?

Подсказка: Один указывает список вариантов, другой — правила выбора по ширине.

Ответ:

- **`srcset` и `sizes`**
- `alt` и `title`
- `width` и `height`

🔮 Шаг 04:

Миссия 4

📄 Сопоставьте элементы `picture` с их назначением.

Подсказка: Рассмотрите, какой элемент задает источник, а какой отображается как резервный.

Ответ:

- **`picture`** — Контейнер для нескольких источников изображений
- **`source`** — Определяет формат изображения и условия его отображения
- **`img`** — Резервное изображение, отображается если `source` не поддерживается

🔮 Шаг 05:

Миссия 5

📄 Сопоставьте атрибуты `source` с их функцией.

Подсказка: Один определяет тип файла, другой — условия отображения.

Ответ:

- **`srcset`** — Список изображений с указанием размеров или плотности пикселей
- **`type`** — Указывает MIME-тип изображения (например `image/webp`)
- **`media`** — Задает условия отображения, аналог `media query`

🔗 Шаг 06:

Миссия 6

Иван хочет, чтобы на мобильных устройствах загружалось меньше данных. Как это сделать с picture?

Подсказка: Браузер выбирает подходящий источник на основе media и srcset.

Ответ:

- **Использовать source с media и srcset для разных размеров**
- Использовать только один img без source
- Использовать CSS background-image вместо picture

🔗 Шаг 07:

Миссия 7

Светлана создает адаптивную галерею. Какой элемент позволяет переключаться между WebP и JPEG автоматически?

Подсказка: Рассмотрите контейнер с несколькими source.

Ответ:

- **picture**
- div
- svg

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте расширения файлов с типами изображений.

Подсказка: Подумайте, какие форматы поддерживают сжатие без потерь и современные браузеры.

Ответ:

- **.webp** — Современный формат с хорошей компрессией
- **.jpeg** — Стандартный формат для фото
- **.png** — Поддерживает прозрачность, без потерь

🔗 Шаг 09:

Миссия 9

📄 Сопоставьте медиа-запросы с типом устройства.

Подсказка: Ширина viewport определяет, какое изображение загружать.

Ответ:

- **(max-width: 600px)** — Мобильные устройства
- **(min-width: 601px) and (max-width: 1200px)** — Планшеты
- **(min-width: 1201px)** — Десктопы

🔗 Шаг 10:

Миссия 10

Дмитрий хочет, чтобы браузер сам выбрал лучший формат изображения. Что нужно использовать?

Подсказка: Рассмотрите комбинацию picture и source с type.

Ответ:

- **picture с source type и резервным img**
- Только img с srcset
- CSS background-image

🔗 **Шаг 11:**

Миссия 11

Наталья хочет оптимизировать загрузку изображений для Retina дисплеев. Что делать?

Подсказка: Используйте srcset с кратными масштабами (1x, 2x).

Ответ:

- **srcset с указанием 1x, 2x**
- Использовать только один размер изображения
- CSS transform scale

🔗 **Шаг 12:**

Миссия 12

📖 Сопоставьте порядок работы браузера с picture с действием.

Подсказка: Браузер проверяет условия и выбирает подходящий источник.

Ответ:

- **Проверяет media source** — Определяет, подходит ли медиа-запрос
- **Проверяет type source** — Определяет, поддерживается ли формат браузером
- **Выбирает подходящий source** — Использует первое удовлетворяющее условие
- **Если нет подходящего source** — Показывает резервное изображение img

🔗 **Шаг 13:**

Миссия 13

📖 Сопоставьте типичные ошибки с их последствиями при использовании picture.

Подсказка: Ошибки обычно связаны с неправильно указанными атрибутами или порядком тегов.

Ответ:

- **Неправильный порядок source и img** — Резервное изображение может быть загружено первым
- **Отсутствие type** — Браузер может не выбрать современный формат
- **Неправильные media** — Загружается не тот размер изображения

🔗 **Шаг 14:**

Миссия 14

Расположите шаги использования picture в правильном порядке:

1. Создать picture контейнер
2. Добавить один или несколько source с srcset и type/media
3. Добавить резервное изображение img
4. Браузер выбирает подходящий source и отображает img, если не найдено подходящего

Подсказка: Сначала создаём контейнер, потом источники, затем резервное изображение, и только после этого браузер делает выбор.

Ответ:

- Создать picture контейнер
- Добавить один или несколько source с srcset и type/media
- Добавить резервное изображение img
- Браузер выбирает подходящий source и отображает img, если не найдено подходящего